



P000340SE PROJECT Report

Product Owner: Branko Minic, Adrian Zielonka

Supervisor: Maria Spichkova

Authors : Cassandra Tedcy Britto, s3922104
Di Wang, s4070702
Gabby Sanchez, s4039866
Jordan Chiou, s4068959
Sneha Thankam Oommen, s4070343

DOCUMENT CONTROL

Version #	Implemented By	Implementation Date	Reviewed By	Approval Date	Reason
1	Di Wang, Jung-De Chiou, Cassandra Tedcy Britto	12/10/2025			Added the report skeleton for each section
2	Di Wang, Jung-De Chiou	20/10/2025			Added Technical Solution Design and old documents (project charter, risk logs)
3	Cassandra Tedcy Britto, Jung- De Chiou, Di Wang	24/10/2025			Test Plan, Technical Solution Design first draft, Development Guide
4	Cassandra Tedcy Britto, Jung- De Chiou, Di Wang	29/10/2025	Maria Spichkova	30/10/2025	First Draft to supervisor for feedback and refinement
5	Jung-De Chiou, Di Wang	02/11/2025			Product Backlog and final adjustment

1. PROJECT REPORT	6
2. PROJECT CHARTER.....	8
2.1. Project Summary	9
2.2. Project Sponsor/Product owners	8
2.3. Stakeholders and End Users	9
2.4. Appointment of Scrum Master	9
2.5. Project Team Members	10
2.6. Project Methodology and Approach.....	10
2.7. Project Governance	11
2.8. Project Scope & Deliverables.....	12
3. ISSUE AND RISK REGISTER.....	16
3.1. Risk Register	16
3.2. Issues Register.....	19
4. PRODUCT BACKLOG	21
4.1. Product Backlog Selection Process	21
4.2. Definition of Done.....	22
4.3. Product Backlog Items	23
4.4. Additional Item Types – Task and PoC	28
5. TECHNICAL SOLUTION DESIGN	30
5.1. Technical Environment	30
5.1.1. UI/UX Design Environment	30
5.1.2. Development Environment	30
5.1.3. Version Control Environment.....	31
5.1.4. Cloud Infrastructure	32
5.1.5. Project Management	33
5.1.6. Hardware and Software Requirements	34
5.2. System Architecture	34
5.2.1. User Interaction Layer	35
5.2.2. API and Application Logic Layer.....	35
5.2.3. Storage and Processing Layer	36
5.2.4. AI and Processing Pipeline	36
5.2.5. Event and Notification Layer.....	36
5.2.6. Data Flow Summary	37
5.2.7. Security and Access Control.....	38
5.2.8. Advantages of the Architecture	38
5.3. Functionalities/features	39
5.3.1. Functionality 1 – Secure Third Party and Email Login	39
5.3.2. Functionality 2 – Preview Editing	42
5.3.3. Functionality 3 – Upload Receipt	49
5.3.4. Functionality 4 – Receipt Text Extraction	52
5.3.5. Functionality 5 – AI Categorisation.....	55
5.3.6. Functionality 6 – Visualisation of Expenses.....	56
5.3.7. Functionality 7 – Time Filtering	61
5.3.8. Functionality 8 – Receipt History	63
5.3.9. Functionality 9 – Single Receipt Detail	66
5.3.10. Functionality 10 – Receipt Detail Editing	68
5.4. Database and Storage Architecture	71

5.4.1.	Database – Amazon DynamoDB	71
5.4.2.	S3 Bucket Structure Design	73
5.5.	Non-functional specifications	76
5.5.1.	Privacy and Data Protection.....	76
5.5.2.	Security	76
5.5.3.	AI Categorisation Accuracy	77
5.5.4.	Performance	77
5.5.5.	Scalability and Reliability	80
5.6.	Known Issues & Limitations	81
5.6.1.	Textract Recognition Limitations	81
5.6.2.	System Limitations	82
6.	TEST PLANNING AND RESULT	85
6.1.	Introduction	85
6.1.1.	Objectives	85
6.1.2.	Tasks	85
6.2.	Test Items	85
6.3.	Features to be tested	86
6.4.	Features not to be Tested.....	94
6.5.	Approach	94
6.5.1.	Code Inspection	94
6.5.2.	Unit Testing	94
6.5.3.	Functional Testing	95
6.5.4.	Test Data	95
6.5.5.	Required Software	95
6.5.6.	Constraints	95
6.5.7.	Logistics	95
6.5.8.	Versioning.....	95
6.6.	Manual Functional Test Cases	96
7.	DEVELOPMENT GUIDE	111
7.1.	Project summary	111
7.2.	Setup client/ developer machine.....	111
7.3.	Code/ comment standard	112
7.4.	Version control policy	112
7.5.	Deployment	112
7.6.	Getting started	115
7.6.1.	How to start.....	115
7.6.2.	Frontend Implementation.....	116
7.6.3.	Backend Implementation	116
8.	REFERENCES.....	118

Project Report

1. PROJECT REPORT

In an increasingly digital and cashless society, individuals often rely on receipts—both paper and digital—to manage their personal finances. However, manual expense tracking remains time-consuming, error-prone, and inefficient. Many existing expense management solutions still depend heavily on manual input, which discourages consistent use.

To address this challenge, ReceiptCAT was developed as an intelligent web-based prototype that leverages artificial intelligence (AI) and cloud technologies to automate receipt scanning, text extraction, and expense categorisation. The project was sponsored by Shine Solutions Group and completed as part of the RMIT Software Engineering Capstone Project (Project ID P000340SE). The solution demonstrates how serverless architectures and AI models can simplify everyday financial management while showcasing scalable and maintainable system design.

The core problem the project addresses is the lack of automation in personal expense tracking and the limited use of AI in consumer-grade receipt management tools.

Manual methods lead to inconsistent data entry, difficulty analysing spending habits, and the loss of valuable budgeting insights. By integrating AWS Textract for Optical Character Recognition (OCR) and AWS Bedrock for AI-driven item categorisation, ReceiptCAT aims to reduce human effort and improve data accuracy.

The study is significant as it explores how serverless and AI-powered systems can enhance financial transparency and encourage sustainable financial behaviour among users, while also serving as a practical case study in applied software engineering.

Project Charter

2. Project Charter

1.1. Project Summary

ReceiptCAT is a web-based application designed to help users categorise and track their shopping expenses by automatically analysing and classifying receipt data. This project only involves designing and implementing a prototype application. The sponsor is offering this project to provide an application that enables more detailed and accurate categorisation of receipts and their expense items (e.g., fresh fruits, cleaning products, party supplies, etc.), making it easier for users to monitor their spending habits and make more informed financial decisions.

The main objectives for the project team are to develop an intuitive and robust web application prototype that allows users to upload receipts and receive accurate categorisation of their purchases, and to generate personal reports that provide better insights into their expenses.

1.2. Project Sponsor/Product owners

The ReceiptCAT project sponsor is [Shine Solutions Group](#), an Australian technology consulting company renowned for its expertise in digital platforms and apps, AWS cloud solutions, and managed cloud services. Founded in 1998, Shine Solutions has earned the trust of major enterprises for over 27 years. The company is an accredited partner with leading cloud providers, including AWS, Google Cloud, Microsoft Azure, and Adobe. Additionally, it is an approved supplier on key government digital marketplaces, such as the Australian Government's Digital Transformation Agency, Buying for Victoria, and buy.nsw.

The project owners for ReceiptCAT include Branko Minic and Adrian Zielonka, both highly experienced industry professionals from Shine Solutions Group. As product owners, they are responsible for prioritising project requirements, providing feedback to guide development, and ensuring that the final deliverables align with the project vision and business objectives. They also serve as the main points of contact for clarifying requirements and making key project decisions.

1.3. Stakeholders and End Users

The key stakeholders for the ReceiptCAT project include:

- **Project sponsors:** Shine Solutions Group, who provide the project opportunity for the team members and is the recipient of the final deliverables.
- **Product owners:** Branko Minic and Adrian Zielonka, who are responsible for providing overall project direction, approving deliverables and acting as technical mentors.
- **Course coordinator:** Dr. Golnoush Abaei (RMIT), who is responsible for coordinating the capstone course, ensuring that the project aligns with academic requirements and supporting students throughout the project lifecycle.
- **Academic supervisor:** Dr. Maria Spichkova (RMIT), who oversees academic standards, ensures alignment with course requirements and assesses student outcomes.
- **Mentors from Shine Solutions:** Branko Minic and Adrian Zielonka, who provide technical guidance, architectural feedback, and ongoing industry-level mentoring.
- **Development team members:** Cassandra Tedcy John Britto, Di Wang, Jordan Chiou, Gabby Sanchez, and Sneha Thankam Oommen, who are the designers and developers that contribute directly to the development and delivery of the application.

The end users of the system are:

General consumers who want to track and categorise their spending using digitised receipts. These users are typically non-technical individuals who value clear, well-structured interfaces and automated tools for expense tracking and budget management.

2.1. Appointment of Scrum Master

The role of Scrum Master will be rotated among team members, with a different person serving as Scrum Master for each sprint. The detailed assignments are as follows:

Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
Cassandra Tedcy John Britto	Gabby Sanchez	Sneha Thankam Oommen	Di Wang	Jordan Chiou

Table 1 Appointment of Scrum Master for each sprint

The primary responsibilities of the Scrum Master include facilitating Scrum processes, managing sprint progress, maintaining the [Trello board](#), hosting meetings, assisting the team in identifying and removing blockers, and ensuring that Scrum best practices are followed.

The development team has decided to follow a rotational Scrum Master approach to encourage shared responsibility within the team. This is because the team operates under a flat hierarchy that fosters transparent collaboration and knowledge sharing. By rotating this role, each member will have the opportunity to understand the responsibilities of a Scrum Master, develop their skills, and participate more deeply in team leadership and project delivery.

2.2. Project Team Members

The project team members and their respective roles are:

Role	Members	Responsibilities
Frontend Developer	Di Wang, Jordan Chiou	Designs the overall UI/UX, develops and maintains the frontend and overall user experience of the web application, and integration with the backend.
Backend Developer	Cassandra Tedcy John Britto (lead), Sneha Thankam Oommen	Designs and implements the backend logic, APIs, database, AI categorisation, and integration with the frontend and external services.
DevOps Engineer	Sneha Thankam Oommen (lead), Cassandra Tedcy John Britto	Sets up and maintains the cloud infrastructure, CI/CD pipelines, and ensures smooth deployment processes.
Solution Architect	Gabby Sanchez	Oversees the overall system architecture, makes key technical decisions, and ensures security and best practices.
QA Engineer	Cassandra Tedcy John Britto, Di Wang, Jordan Chiou, Gabby Sanchez, Sneha Thankam Oommen	Writes unit and integration tests on all relevant code and ensures the test coverage is 85% or higher.
Scrum Master	Sprint 1 – Cassandra Tedcy John Britto Sprint 2 – Gabby Sanchez Sprint 3 - Sneha Thankam Oommen Sprint 4 – Di Wang Sprint 5 – Jordan Chiou	Facilitates scrum processes, coordinates Scrum meetings, and assists in removing obstacles for the team.

Table 2 The Development Team Roles

Note: The roles do not imply that the assigned team member is solely responsible for that part. Other members can share the responsibilities.

2.3. Project Methodology and Approach

The methodology adopted for this project is the **Scrum-based Agile Methodology**. This approach was chosen because it is currently the most widely accepted methodology in the industry. Additionally, since the team is building a new product, frequent iterations, rapid feedback, and adaptability to changes are essential. Scrum enables the team to deliver incremental improvements, gather client input regularly, and make continuous adjustments to meet project goals. For details on the role and responsibilities of the Scrum Master, please refer to [Section 2.4](#).

The team will meet with the client and supervisor every Monday at 11:30 a.m. to conduct weekly progress reviews and provide feedback using Microsoft Teams. Additionally, the team will hold at least two fixed internal meetings each week (Mondays and Fridays), in a hybrid format based on members' availability. Online meetings will be conducted via Slack, while in-person meetings will take place in a pre-booked private meeting room. Furthermore, Slack will also serve as the primary channel for ongoing communication with the client, enabling the quick resolution of questions and issues.

The overall project timeline consists of a Sprint 0 (for initial planning), followed by five development sprints, each lasting 2 weeks, for a total of 12 weeks of development. The final deliverables will be

presented during the final presentation in the 13th week. During Sprint 0, the team will prepare the Product Backlog (prioritised according to client requirements) and set up the project environment and development tools. Story points will be used to estimate the time required to develop each backlog item until completion, with one story point equivalent to four hours of work.

Throughout each sprint, the Scrum Master will facilitate the following Scrum meetings:

- **Retrospective Meeting:** Held fortnightly, at the end of each sprint, to review what went well, what didn't, and what can be improved. Outcomes and actionable points will be recorded as sprint retro notes and used to optimise the next sprint.
- **Sprint Planning Meeting:** Held fortnightly, also at the end of each sprint (often immediately after the retrospective if available). The team will plan the next sprint, taking into account improvements identified during the retrospective, backlog item priorities, estimated story points, and team velocity. New, reworked, or delayed items will be discussed and planned accordingly. Sprint planning notes will be documented for traceability.
- **Standup Meeting:** Held three times a week (Monday, Wednesday, and Friday), via Slack messages instead of traditional daily standups. Given that team members also have academic and work commitments, the team has agreed to perform this meeting asynchronously. Members will share updates on their progress, plans, and any roadblocks in the Slack channel on every assigned day.

Project management will be conducted using Trello to plan, track, and review sprint progress and product backlog items. Trello was selected as the project management tool after discussions with the supervisor and client approval, as it is well-suited to a project of this scale and provides transparency and collaboration for all stakeholders.

2.4. Project Governance

The Project Governance model is as follows:

ReceiptCAT follows a collaborative agile governance to support flexibility, accountability, and transparency. The team utilises Slack as its primary platform for daily communication, progress updates, and resource sharing. All team members, along with academic supervisor Dr. Maria Spichkova and Shine Solutions mentors Adrian Zielonka and Branko Minic, have been added to both the Slack workspace and the Trello board to maintain visibility and alignment.

Project decisions are made democratically, and Slack polls are used to facilitate voting and reach a consensus. Formal internal team meetings are held twice a week, either in-person or via Slack. Additionally, asynchronous stand-up meetings are held three times a week (Monday, Wednesday, and Friday) via Slack messages, enabling the team to discuss progress, resolve blockers, and stay on track.

The team follows Scrum-based Agile methodology and has chosen to rotate the Scrum Master role across five sprints to encourage equal participation and responsibility. They will be responsible for leading daily stand-ups, identifying and resolving blockers with the team, and ensuring that Scrum practices are followed. Please refer to [Section 2.4](#) for detailed information on the Scrum Masters chosen for each Sprint, and their roles and responsibilities.

Responsibilities within the project are clearly defined, with each member playing a specific role to ensure the project's success.

- Di Wang and Jordan Chiou are responsible for the frontend.
- Cassandra Tedcy John Britto (Lead) and Sneha Thankam Oommen are responsible for backend development.
- Sneha Thankam Oommen (Lead) and Cassandra Tedcy John Britto are responsible for handling DevOps.
- Gabby Sanchez is the Architect.

Please refer to [Section 2.5](#) for detailed information about the roles.

The team will operate collaboratively; that is, all members will contribute across different areas, irrespective of their responsibilities, offer support, and assist one another wherever needed to ensure continuous learning and delivery.

To support agile delivery, the team has created a product backlog populated with user stories, which help break down features into manageable, testable tasks. Sprint planning, milestone tracking, and retrospectives are managed via Trello to ensure visibility, adaptability, and continuous improvement throughout the project.

A [Section 3.1](#) is in place to proactively identify and address challenges. It contains detailed technical and non-technical risks, their mitigation strategies, and contingency plans. At a high level, risks are first addressed within the team. If unresolved, they are escalated to Dr. Maria Spichkova for academic oversight and, if necessary, further to Shine Solutions mentors, Branko Minic or Adrian Zielonka, for industry-level guidance.

An [Section 3.2](#) is maintained to record and manage all issues that arise during the project lifecycle. It captures details such as the issue description, category (technical or non-technical), owner, impact, resolution actions, and current status. Each issue is reviewed during internal meetings and sprint retrospectives to ensure accountability and timely resolution. Issues are first handled collaboratively within the team. If an issue cannot be resolved internally or impacts deliverables, it is escalated to the academic supervisor, Dr. Maria Spichkova, for guidance. For issues requiring technical or architectural decisions, further escalation is made to the Shine Solutions product owners, Branko Minic and Adrian Zielonka, for industry-level support and advice.

Scope changes are tightly controlled to avoid scope creep. Any proposed change must be discussed during the meetings and approved by all members before being added to the backlog.

2.5. Project Scope & Deliverables

Scope:

The scope of the ReceiptCAT project is to design and build a prototype full-stack web application that enables users to manage their spending through digitised receipts.

The scope covers all pre-requisite items (Proof of Concepts) and Product Backlog Items (PBIs), prioritised as High, Medium, and Low. Pre-requisite items and high-priority PBIs are targeted for delivery in early sprints, while Medium and Low PBIs will be delivered in later sprints.

Below are the key product features along with their functionalities:

- User Management: Sign up and log in via email/password or third-party authentication.
- Receipt Upload:
 1. Upload a single receipt image (JPG/PNG) from desktop or mobile.
 2. Preview, reselect, or cancel a selected image before uploading.
 3. Crop or blackout selected receipt images before uploading.
 4. Display the receipt upload progress after the upload completes.
 5. Handle errors for unsupported file types/sizes.
- Receipt Categorisation:
 1. Extract text representing expense items from receipts.
 2. Classify expense items into specific categories.
- Dashboard:
 1. View purchase details, including name, quantity, and amount.
 2. Provide a time range filter. The default range is set to monthly and grouped by category, but users can also select yearly or other custom time ranges as needed.
 3. Display an overview of all receipts uploaded within the selected time range, including the total cost, item number and receipt. The default range is set to the current month
- Receipt History:
 1. View every receipt summary, including vendor name, item number, total cost and purchase date.
 2. Display an overview of all receipts, including total cost, item number and receipt counts.
- Receipt Detail:
 1. View purchase details, including vendor name, item count, item name, quantity, amounts, purchase date and category.
 2. View uploaded receipt files.
- Manual Data Management: Edit vendor name, item names, categories, item counts, and unit prices. Add new items or delete existing items as needed.

Deliverables:

The deliverables outlined below represent the agreed outputs expected by the key stakeholders. They cover both technical and non-technical aspects of the project, ensuring that the final solution meets functional, operational, and documentation requirements.

Below are the technical deliverables that are planned to be completed and delivered in successive iterations:

- **Frontend:** Fully functional and responsive web application in React with TypeScript, integrated with AWS Cognito authentication and backend APIs, deployed using AWS CloudFront and S3.
- **Backend:** Serverless backend using AWS Lambda (TypeScript) accessed via Amazon API Gateway; integrated with AWS Textract for OCR extraction and AWS Bedrock for AI-driven item categorisation and performance evaluation; uploaded receipts stored in AWS S3, and extracted receipt data stored in AWS DynamoDB.
- **Infrastructure:** Infrastructure as Code using AWS CloudFormation; CI/CD pipelines using AWS CodeBuild and CodePipeline for automated build, test, and deployment, ensuring consistent infrastructure provisioning and automated testing across environments.
- **Testing:** Unit testing with Jest, and integration (end-to-end) testing with Cypress, maintaining at least 80% overall coverage for the frontend and backend components.
- **Source code:** Three private repositories (Frontend, Backend, and Infra) created and hosted in AWS CodeCommit under the account owned by Shine Solutions Group.

Below are the non-technical deliverables:

- Bedrock & Textract Evaluation Report
- Approved Product Backlog hosted in Trello, with prioritised PBIs.
- Project Charter
- Risk/Issues Register Document
- Technical Solution Design
- Development Guide
- Test Planning and Results
- Sprint artefacts (Sprint Planning notes, Sprint Review notes, Sprint Retro notes).
- Client meeting agendas and minutes
- Team meeting minutes
- Analysis of ethical considerations and Learning outcomes
- Presentation slides

The project summary, scope and deliverables will be reviewed and signed off by the product owners at Shine Solutions, and the entire Project Charter will be reviewed and signed by the Academic supervisor. The final product must meet both academic and industry benchmarks.

Risk and Issue Register

3. Issue and Risk Register

3.1. Risk Register

The Risk Register identifies and documents potential risks that could affect the project's progress, cost, or quality. It includes a description of each risk, its assessed impact and likelihood, the responsible owner, and the strategies in place to mitigate and respond to these risks if they occur.

No	Description	Impact	L'hood	Owner	Mitigation Strategies	Contingency Plan
1	AWS budget overrun due to uncontrolled usage of AWS services (e.g., Bedrock, RDS)	H	M	All Developers	Set service quotas; Choose cost-effective AI and DB services; Utilize high-cost services after stakeholder approval	Disable expensive services when not in use; Decrease provisioned settings such as instance sizes and concurrency limits
2	Low test coverage or unstable tests resulting in pipeline failures	M	M	All Developers	Set higher test coverage in local environment, review test quality during code reviews, use pair-programming on complex tests; Write non-flaky tests	Isolate failing tests; Log technical debt and assign refactoring tasks
3	Inconsistent receipt categorisation due to weak AI/ML model generalisation	M	M	Backend / Architect	Validate and review training data, test model with varied example data	Implement manual user override/edit functionality; Re-train with more examples; Downgrade to a simpler categorisation ruleset
4	CI/CD failures or misconfiguration	H	M	DevOps/Architect	Conduct POCs early and identify best practises	Revert to manual configuration and deployment until failures are fixed.
5	Unauthorised access to AWS services	H	M	DevOps/Architect	Ensure least-privilege access to all deployed services and roles.	Block service until the unauthorised access has been identified through trace logs and appropriate action has been taken.
6	Unauthorised access to saved data	H	M	All Developers	Data is accessed using pre-signed URLs. Enable encryption at rest and in transit. Remove unnecessary PII data before saving into storage.	Identify securities vulnerabilities ASAP and resolve.
7	Poor image quality leading to extract text failure	M	H	Frontend / Backend Developers	Add image upload guidelines (provide example images for reference before upload); implement a resolution check before upload is accepted; reject	Provide manual edit option for users to input data, log failed extract text cases for internal review, allow user to re-upload after the failure.

					files below a resolution threshold and prompt the user to re-upload a clearer image.	
8	UI components may not render correctly across different browser or screen sizes.	M	M	Frontend Developer	Use responsive design and cross-browser testing tools; follow accessibility guidelines.	Add browser-specific fixes where needed, and document known compatibility issues to prevent future confusion or duplicate bug reports.
9	Large receipt files or processing delays may cause Lambda timeouts or memory errors	H	M	Backend Developer/Architect	Optimize Lambda functions; split tasks into smaller steps; monitor usage.	Log failures, notify user, retry the process with a simpler setup (like reducing the file size or skipping some steps)
10	SNS notification does not reach user or is delayed.	M	L	Backend Developer/Architect	Checking if user subscriptions are proper and SNS is setup correctly. Testing notification flow regularly	Retry to send notification, update the receipt status on the dashboard so the user can check manually.
11	Integration failure between backend and frontend.	H	M	All Developers	Test integration regularly and clearly define API contracts during the initial stages of development.	Display fallback message to users, log issues, and coordinate to fix the issue quickly.
12.	Team member unavailable during a sprint due to an illness or other personal reasons.	M	M	Scrum Master	Stay up to date with what each team member is working on, document progress in detail, and regularly share knowledge.	Reassign tasks within 1 working day to maintain sprint velocity and adjust the sprint scope.
13.	Requirement changes or scope creep during the project lifecycle	H	L	PO/ Scrum Master	Set up regular meetings with stakeholders to clarify requirements; use a change control process for any requested changes; keep documentation up to date	Re-assess and re-prioritize backlog, communicate impacts on timeline and deliverables to all stakeholders, and negotiate scope where necessary
14.	Failure or unavailability of external APIs or third-party services (e.g., Textract, Auth0, Cognito)	H	M	Backend Developer / Architect	Monitor third-party service health, design with fallback logic or retries, and regularly review SLAs	Switch to a backup provider, if possible, notify users about the issue, and implement manual processing, if necessary, until the service is restored
15.	Incomplete or outdated project documentation making onboarding or maintenance difficult	M	M	All Developers	Allocate time each sprint for documentation updates; conduct documentation reviews; ensure onboarding materials are available	Schedule knowledge transfer sessions, assign documentation tasks to specific team members, and pair new members with experienced developers for support

16.	Group not being able to reach a consensus	H	M	All Developers	Use structured decision-making frameworks; Assign a rotating facilitator role to guide discussions and ensure balanced participation; Document decisions and open issues in Trello to avoid re-discussing resolved points.	Allocate a maximum 30 minutes in a meeting to resolve the issue to attempt resolution; If still unresolved, defer to next scheduled meeting with mediator, Contact Dr. Maria Spichkova first, the supervisor of the project. If further clarification is required, Shine solution mentors Branko Minic or Adrian Zielonka can be contacted, who can provide the team with architectural feedback and industry-level guidance.
-----	---	---	---	----------------	--	---

Table 3 Risk Register

Impact = Impact on the Business or project in the event the risk is realized

L'hood = Likelihood of risk being realized

3.2. Issues Register

The Issues Register records problems and obstacles that arose during the project's execution, including technical, operational, and coordination-related issues. Each entry documents the issue description, its impact, the allocated members, the date, and the resolution/action taken by the development team.

No	Description	Impact	Date Raised	Phase Raised	Allocated to	Resolution Strategies / Actions	Date Closed
1	Accidental deletion of AWS resources due to inconsistent naming conventions.	M	17 Aug 2025	Sprint 1	Jordan Chiou	Established strict naming protocols and centralised infrastructure configuration within CodeCommit; validated naming rules in CI/CD pipeline	18 Aug 2025
2	Uneven task distribution and under-burn during sprint planning	M	22 Aug 2025	Sprint 1	Scrum Master (Cassandra Tedcy John Britto)	Improved task estimation and created backlog refinement sessions for even allocation	25 Aug 2025
3	Incorrect Git merge: changes from develop were accidentally merged into a feature branch, causing version conflicts	M	31 Aug 2025	Sprint 2	Sneha Thankam Oommen	A new branch feature2 was created to preserve only the intended changes. Team members redirected ongoing work to the new branch. Improved branch naming and merge review process to prevent similar errors.	01 Sep 2025
4	Unit and E2E tests not following the AAA pattern	M	04 Sep 2025	Sprint 2	Frontend team	Refactored all tests to follow AAA structure; added testing checklist in PR reviews	19 Sep 2025
5	Difficulty setting up mocks for Jest and Cypress tests	M	05 Sep 2025	Sprint 2	Di Wang	Created shared mock utility functions; conducted internal workshop for testing best practices	19 Sep 2025
6	UI layout on desktop view	M	19 Sep 2025	Sprint 3	Frontend team	Redesigned layout and improved responsive grid system	23 Sep 2025
7	Increased workload and fatigue during presentation preparation	L	07 Oct 2025	Sprint 5	Scrum Master	Added buffer time in sprint	19 Oct 2025

Table 4 Issue Register

Impact = Impact on the Business or project in the event the issue is not resolved within the required time.

PRODUCT BACKLOG

4. Product Backlog

4.1. Product Backlog Selection Process

The Product Backlog (PB) was collaboratively developed and prioritised by the development team and the product owner at the beginning of the project. Backlog items were initially derived from the client's functional requirements and refined during the sprint planning sessions. Each item was written in the user story format — “As a [role], I want [goal], so that [benefit]” — to ensure clarity and alignment with end-user value.

During backlog grooming, the team assessed each Product Backlog Item (PBI) based on business value, technical complexity, and dependency impact. All high- and medium-priority PBIs were completed within the project timeframe, covering the core system functionalities such as secure authentication, receipt upload and preview, AI-based text extraction and categorisation, and dashboard analytics. Only several low-priority PBIs — primarily optional features such as profile customisation, export options, and additional visualisation types — remained incomplete due to the client's decision to prioritise essential deliverables and the limited project duration.

Each PBI was assigned an Effort Point, representing the estimated development workload, including both the time for technical research and the actual implementation effort. One Effort Point was equivalent to approximately four hours of development work. Each PBI was also supported by corresponding Acceptance Criteria (ACs), Development Tasks, and QA Tasks, all documented and tracked on the [ReceiptCAT Trello board](#).

4.2. Definition of Done

The Definition of Done (DoD) was collectively established by the development team to ensure consistency and quality across all deliverables. After internal discussion and review, the proposed DoD was confirmed with the product owner to align on delivery expectations and acceptance standards.

A backlog item was marked as Done only when it met all acceptance criteria, passed the corresponding unit and integration tests, and was successfully deployed to the development environment without defects.

The agreed DoD included the following criteria:

- Unit Testing completed (coverage 80%)
- Peer Reviewed
- Functional testing based on Acs completed
- Usability testing passed in Google Chrome version 134.0 or later, on both Mobile 1080p or higher, and Desktop 1080p or higher screen sizes
- Regression testing completed
- Approved by the Product Owner

This standardised DoD ensured that each completed PBI contributed to a production-ready, stable system increment and that all delivered features met both technical and business quality expectations.

DEFINITION OF DONE	
#	Description
1	Unit Testing completed (coverage 80%)
2	Peer Reviewed
3	Functional testing based on Acs completed
4	Usability testing passed in Google Chrome version 134.0 or later, on both Mobile 1080p or higher, and Desktop 1080p or higher screen sizes
5	Regression testing completed
6	Approved by the Product Owner

Table 5 Definition of Done

4.3. Product Backlog Items

The table below lists all Product Backlog Items (PBIs) defined and maintained during the project lifecycle.

Each entry records the unique ID, description, assigned owner, priority level, estimated effort, current status, and associated Definition of Done reference.

The backlog was actively tracked and updated in the team's Trello board to support sprint planning, progress monitoring, and retrospective reviews.

ID	ITEM	OWNER	PRIORITY	EFFORT	STATUS	DEFINITION OF DONE
PBI-001	As a user, I want to view a landing page with a clear option to sign up or log in, so that I can get started easily.	PO - Branko Minic	High	1	Done	DoD1
PBI-002	As a registered user, I want to login using my credentials, so that I can access my dashboard and data.	PO - Branko Minic	High	3	Done	DoD1
PBI-003	As a new user, I want to sign up with my desired credentials, so that I can create an account and access the system.	PO - Branko Minic	High	3	Done	DoD1
PBI-004	As a logged-in user, I want to select a single receipt image (JPG or PNG) so that I can proceed to review it before uploading.	PO - Branko Minic	High	2	Done	DoD1
PBI-005	As a logged-in user, I want to preview the image before uploading, so that I can check if it is the right file I intend to upload.	PO - Branko Minic	High	2	Done	DoD1
PBI-006	As a logged-in user, I want to cancel the selected image before uploading so that I can exit the upload process without making any changes.	PO - Branko Minic	High	1	Done	DoD1
PBI-007	As a logged-in user, I want to reselect a different image before uploading, so that I can submit the correct image.	PO - Branko Minic	High	2	Done	DoD1

PBI-008	As a logged-in user, I want to view the progress of the receipt upload, so that I can wait for the process to finish confidently.	PO - Branko Minic	Medium	3	Done	DoD1
PBI-009	As a logged-in user, I want to view a list of receipt summaries on my Receipt History page so that I can quickly review my past purchases.	PO - Branko Minic	High	3	Done	DoD1
PBI-010	As a logged-in user, I want my receipts to be automatically assigned to predefined basic categories, so that I can quickly understand my spending patterns.	PO - Branko Minic	High	5	Done	DoD1
PBI-011	As a logged-in user, I want to view a detailed page showing the summary of the receipt and all its extracted expense items so that I can review the full details of a specific purchase.	PO - Branko Minic	High	3	Done	DoD1
PBI-012	As a logged-in user, I want to view the images of the uploaded receipts in the receipt's details page, so I can verify my receipt upload.	PO - Branko Minic	High	5	Done	DoD1
PBI-013	As a logged-in user, I want my expense items to be displayed and grouped by their assigned categories, so that I can easily review and plan my budget.	PO - Branko Minic	Medium	5	Done	DoD1
PBI-014	As a logged-in user, I want to see the total amount spent in each category, so that I can better understand my spending distribution.	PO - Branko Minic	Medium	1	Done	DoD1
PBI-015	As a logged-in user, I want to view my monthly expenses grouped by category on the dashboard, so that I can easily track and check my spending patterns.	PO - Branko Minic	Medium	2	Done	DoD1
PBI-016	As a logged-in user, I want to switch to previous months on the dashboard, so that I can view my expense records from past months.	PO - Branko Minic	Medium	1	Done	DoD1
PBI-017	As a logged-in user, I want to see my total expenses for the month, so that I can keep track of my overall spending.	PO - Branko Minic	Medium	1	Done	DoD1

PBI-018	As a logged-in user, I want to see all individual expense items under each category for a specified month, so that I can review my past purchases and their details.	PO - Branko Minic	Medium	1	Done	DoD1
PBI-019	As a logged-in user, I want the dashboard to display a visual chart of my current month's expenses by category so that I can quickly understand my spending distribution.	PO - Branko Minic	High	3	Done	DoD1
PBI-020	As a logged-in user, I want the option to view my expense data in different chart formats so that I can analyse my spending in the most suitable way.	PO - Branko Minic	Low	3	To Do	DoD1
PBI-021	As a logged-in user, I want the charts on my dashboard to update based on my selected time range so that I can analyse my spending over different time frames.	PO - Branko Minic	Medium	2	Done	DoD1
PBI-022	As a logged-in user, I want to edit the details of each expense item on the receipt details page, so that I can manually correct any incorrect information as needed.	PO - Branko Minic	Medium	3	Done	DoD1
PBI-023	As a logged-in user, I want to delete a receipt record and its associated expense items from the dashboard, so that I can remove unwanted records.	PO - Branko Minic	Low	2	To Do	DoD1
PBI-024	As a logged-in user, I want to manually edit the category of an item, so that I can ensure that my spending breakdown is accurate.	PO - Branko Minic	Medium	2	Done	DoD1
PBI-025	As a logged-in user, I want to view and download the list of all receipts I uploaded to the system, so that I can have a historical record of my purchases.	PO - Branko Minic	Low	3	To Do	DoD1
PBI-026	As a logged-in user, I want to take a photo of a receipt and upload it directly from my phone through the website so that I can quickly digitise my expenses.	PO - Branko Minic	Medium	5	Done	DoD1
PBI-027	As a user, I want to view an overview of ReceiptCAT's features on the home page so that I can understand the benefits of using the application.	PO - Branko Minic	Low	2	To Do	DoD1

PBI-028	As a logged-in user, I want to view my expense listings by year, so that I can review my annual spending trends.	PO - Branko Minic	Deleted	2	To Do	DoD1
PBI-029	As a logged-in user, I want to export the current view of my dashboard to a downloadable PDF file so that I can review the information outside of the application.	PO - Branko Minic	Low	2	To Do	DoD1
PBI-030	As a logged-in user, I want to upload a receipt in PDF format so that I can store my receipts in this file type.	PO - Branko Minic	Low	3	To Do	DoD1
PBI-031	As a logged-in user, I want to upload multiple receipt files at once, so that I can save time when processing many receipts.	PO - Branko Minic	Low	5	To Do	DoD1
PBI-032	As a logged-in user, I want to navigate from the dashboard to my profile page so that I can easily access my personal information and settings.	PO - Branko Minic	Low	2	To Do	DoD1
PBI-033	As a logged-in user, I want to edit my profile details so that my account information remains accurate and current.	PO - Branko Minic	Low	2	To Do	DoD1
PBI-034	As a logged-in user, I want to upload an avatar so that I can visually represent my account.	PO - Branko Minic	Low	2	To Do	DoD1
PBI-035	As a logged-in user, I want to see my avatar displayed on the dashboard and header so that I can easily identify my account.	PO - Branko Minic	Low	2	To Do	DoD1
PBI-036	As a logged-in user, I want to change my avatar, so that I can keep it up to date.	PO - Branko Minic	Low	2	To Do	DoD1
PBI-037	As a new user, I want to receive and enter a verification code sent to my email, so that I can confirm my email address and ensure my account is securely registered to the correct email ID.	PO - Branko Minic	Low	2	Done	DoD1

PBI-038	As a registered user, I want to reset my password via a secure email link, so that I can regain access to my account if I forget it.	PO - Branko Minic	Low	2	Done	DoD1
PBI-039	As a logged-in user, I want the system to detect duplicate receipt uploads so that I can avoid storing the same purchase multiple times.	PO - Branko Minic	Low	2	To Do	DoD1
PBI-040	As a new user, I want to be able to sign up through an alternate set of credentials, so that I can easily access the system.	PO - Branko Minic	Low	3	To Do	DoD1
PBI-041	As a user, I want to crop or blackout selected receipt images before upload, so that I can avoid uploading sensitive information and ensure only the relevant receipt content is processed.	PO - Branko Minic	Medium	5	Done	DoD1

Table 6 Product Backlog

4.4. Additional Item Types – Task and PoC

In addition to standard PBIs, the team introduced two supplementary item types — Task and Proof of Concept (PoC) — to manage project operations and technical exploration activities better.

- Task:

Tasks were used to represent necessary non-functional or non-development work items that directly supported project execution. Examples include environment setup, CI/CD configuration setting adjustment, documentation writing, and administrative work. Categorising these items separately ensured better workload visibility and accurate sprint velocity tracking.

- Proof of Concept (PoC):

PoC items were created to evaluate the feasibility of specific technologies or implementation approaches before committing to full development. This included testing AWS service integrations (e.g., Textract vs. Bedrock model combinations), evaluating UI frameworks, or comparing alternative libraries.

Conducting PoCs early allowed the team to identify technical risks, validate assumptions, and avoid unnecessary time or resource expenditure during actual PBI implementation.

Detailed records of all Task and PoC backlog items can be found in the accompanying document P000340SE-ProductBacklog.xlsx included in Submission 3.

TECHNICAL SOLUTION DESIGN

5. Technical solution design

This subdocument outlines the overall technical architecture, system functionalities, and implementation design of the ReceiptCAT application. It details how the system integrates multiple AWS-managed services—such as Cognito, API Gateway, Lambda, Textract, Bedrock, S3, and DynamoDB—to deliver a fully serverless, AI-powered receipt management solution.

The design prioritises scalability, security, and user privacy while ensuring accurate AI-driven text extraction and expense categorisation. Each subsequent subsection elaborates on the system's architecture, database schema, feature workflows, and performance characteristics, providing a comprehensive view of the implemented technical solution.

5.1. Technical Environment

The purpose of this section is to provide a comprehensive view of the technologies, tools, and platforms selected by the development team, along with the rationale behind each choice, ensuring that all design decisions align with the mandatory requirements and architectural standards defined by the product owner.

5.1.1. UI/UX Design Environment

For UI/UX design, the team used Figma and Adobe Illustrator. Figma was selected for creating both desktop and mobile mock-ups, ensuring consistency across responsive breakpoints and enabling real-time, collaborative iteration among team members. Its component-based structure closely aligns with React's frontend architecture, enabling a direct translation of visual components into reusable code modules. Adobe Illustrator was used for brand identity and logo design, offering precise vector-based control and scalability across web, presentations, and print. Together, these tools provided a cohesive design environment that supported rapid prototyping, consistent visual branding, and clear communication between designers and developers—forming a key part of the project's overall technical solution design.

5.1.2. Development Environment

The development environment was established using modern, industry-standard technologies to ensure scalability, maintainability, and alignment with the project's serverless architecture requirements. Each selected tool and framework was evaluated by both the development team and the product owner for suitability, performance, and ease of integration.

The key components of the environment are summarised below:

- **Frontend:** React using TypeScript

Justification: Evaluated by the development team and product owner as the most suitable standard framework for developing a responsive and modular progressive web application. React enables efficient component-based design and intuitive state management, while TypeScript provides static typing to enhance maintainability, reliability, and error prevention during compilation.

- **Backend:** AWS Lambda functions written in TypeScript

Justification: Evaluated by the development team as the recommended approach to fulfil the serverless architecture requirement specified by the product owner. TypeScript ensures consistency across the full stack and promotes a unified codebase, simplifying integration and reducing runtime issues. Using AWS Lambda supports scalability, cost efficiency, and reduced operational overhead.

- **Testing:**

- **Jest** for unit testing

- Justification: Evaluated by the development team as the preferred unit testing framework for TypeScript projects across both frontend and backend. Jest provides strong TypeScript integration and reliable test isolation, enabling automated verification of business logic and maintaining high code coverage.

- **Cypress** for integration and end-to-end testing

- Justification: Evaluated by the development team as the most appropriate end-to-end testing framework for web applications. Cypress allows realistic browser-based testing, enabling verification of complete user interactions, data flow, and UI rendering.

- **IDE:** Visual Studio Code

Justification: Evaluated by the development team as the recommended integrated development environment for all layers of the application. Visual Studio Code offers built-in TypeScript support, a rich plugin ecosystem, and seamless integration with AWS Toolkit, facilitating local development, testing, and deployment. Additionally, it is free to use and cross-platform, making it suitable for all developers within the project.

5.1.3. Version Control Environment

To ensure collaborative development, source code integrity, and deployment traceability, the project adopted a structured version-control environment built on AWS's native DevOps toolchain.

The following components and practices were implemented to maintain consistency, security, and efficient workflow management across the team:

- **Repository Platform:** AWS CodeCommit

- Justification: Used for secure version control of all project source code. Hosted within the product owner's AWS account, ensuring controlled access through IAM policies.

- **Repository Structure:** Three private repositories were established — frontend, backend, and infrastructure — to maintain modular separation and simplify CI/CD integration.

- **Branching Model:** Git-based feature branching workflow with mandatory code reviews before merging to the main branch.

- **Continuous Integration and Deployment:** Implemented using AWS CodeBuild and CodePipeline to build, test, and deploy code upon commit automatically.

- **Infrastructure as Code (IaC):** CloudFormation templates used to define infrastructure declaratively, ensuring reproducibility and version tracking of cloud resources.

5.1.4. Cloud Infrastructure

The cloud infrastructure for the ReceiptCAT system was designed and deployed entirely within Amazon Web Services (AWS) to meet the product owner's requirement for a fully serverless, scalable environment. This architecture leverages AWS-managed services to deliver high availability, strong security, and seamless integration across all system components—from frontend hosting and authentication to backend execution and AI processing.

The key infrastructure components and their justifications are outlined below:

- **Platform:** Amazon Web Services (AWS)

Justification: The product owner specified it as a mandatory requirement to leverage AWS-native services and implement a fully serverless architecture. AWS provides the scalability, reliability, and integration capabilities necessary to support the system's performance and security goals.

- **Frontend Hosting:** Amazon S3 and CloudFront

Justification: Evaluated by the development team and product owner as the most suitable approach for hosting React applications within AWS. Amazon S3 serves as a secure and cost-efficient static website host, while CloudFront distributes the content globally with caching and HTTPS support to ensure low latency and an optimal user experience.

- **Backend Execution:** API Gateway and AWS Lambda

Justification: Evaluated by the development team as the recommended configuration to achieve a scalable and cost-effective serverless backend. AWS Lambda provides event-driven compute functions that automatically scale based on user demand, and API Gateway exposes RESTful endpoints securely to the frontend.

- **Authentication and User Management:** Amazon Cognito

Justification: Evaluated by the development team and product owner as the preferred service for managing all authentication and authorisation processes, including user registration, login, and third-party identity federation (e.g., Google Sign-In). Cognito enables secure session handling and token-based access control, ensuring user data isolation and compliance with best-practice security standards.

- **Notification and Integration Services:** Simple Notification Service (SNS)

Justification: Adopted as a lightweight messaging and event notification service to enable asynchronous communication between distributed components where required. This supports decoupled integration and enhances system responsiveness.

- **Storage and Database:** Amazon S3 and DynamoDB

Justification: Both were evaluated by the development team as the optimal AWS-native solutions for data persistence. S3 securely stores receipt images and extracted text data, while DynamoDB manages structured and semi-structured metadata with high throughput and low latency. Both services are fully serverless and integrate seamlessly with Lambda functions.

- **AI and Machine Learning Processing:** AWS Textract and AWS Bedrock

Justification: Evaluated and approved by the product owner as the recommended AI services for intelligent document processing and categorisation. AWS Textract extracts key data from receipt images using OCR, while Bedrock enables semantic analysis and expense categorisation powered by large language models.

- **CI/CD Pipeline:** AWS CodeBuild and CodePipeline

Justification: The development team evaluated native AWS DevOps tools as the best fit for implementing continuous integration and continuous deployment workflows. These services automate the build, test, and release processes, ensuring consistency, traceability, and rapid delivery.

- **Infrastructure as Code (IaC):** AWS CloudFormation

Justification: Required by the product owner and evaluated by the development team as the most appropriate tool for managing cloud resources through version-controlled templates. CloudFormation enables consistent, auditable, and repeatable deployments across multiple environments.

5.1.5. Project Management

Effective project management and communication practices were essential to coordinating tasks, maintaining transparency, and ensuring timely delivery throughout the development lifecycle.

A combination of agile management tools and communication platforms was selected to support collaboration between the development team, supervisor, and product owner.

The tools and their justifications are outlined below:

- **Task Management:** Trello

Justification: Used as the project's agile management board to visualise sprint progress and backlog tasks. Recommended by the supervisor and approved by the product owner for suitability for small-team projects.

- **Team Communication:** Slack

Justification: Used as the primary communication channel for daily discussions, task coordination, and issue tracking among team members. It is also the preferred text-based communication platform for the product owner to provide quick updates, clarifications, and technical queries throughout the development process.

- **Client and Supervisor Meetings:** Microsoft Teams

Justification: Utilised as the official platform for scheduled progress meetings, sprint reviews, and stakeholder presentations. Both the development team and the product owner use Microsoft Teams to facilitate weekly discussions and maintain transparency on project milestones.

- **Formal Communication:** Email

Justification: Serves as the formal medium for documentation exchange, milestone reporting, and communication with the supervisor and product owner. It ensures proper record-keeping of approvals, deliverables, and official correspondence.

5.1.6. Hardware and Software Requirements

This section outlines the hardware and software requirements for developing and deploying the ReceiptCAT system. As the project is built on a fully serverless AWS architecture, the hardware requirements for developers are minimal, with most workloads executed through managed cloud services.

The following summarises the key environments and dependencies used throughout the project:

- **Local Development:** Conducted on personal workstations running macOS or Windows. Each setup includes Node.js, AWS CLI, TypeScript, and Visual Studio Code for consistent development across environments.
- **Cloud Deployment:** All production workloads run on managed AWS services provided by the product owner, eliminating the need for on-premise hardware.
- **Software Dependencies:** All development tools and frameworks used (React, Jest, Cypress, etc.) are open source or included within AWS-provided toolkits.
- **Hardware Requirements:** No specific hardware requirements beyond standard developer laptops, as the application operates entirely within the cloud.

5.2. System Architecture

The ReceiptCAT system adopts a serverless architecture hosted entirely on Amazon Web Services (AWS). This architecture was chosen to achieve high scalability, low operational overhead, and seamless integration with AI services for text extraction and categorisation. The system enables users to upload receipt images, which are automatically processed, categorised into expense groups, stored securely, and presented back to the user through a responsive web interface.

Please refer to Figure 1 below for a visual representation of the overall system design.

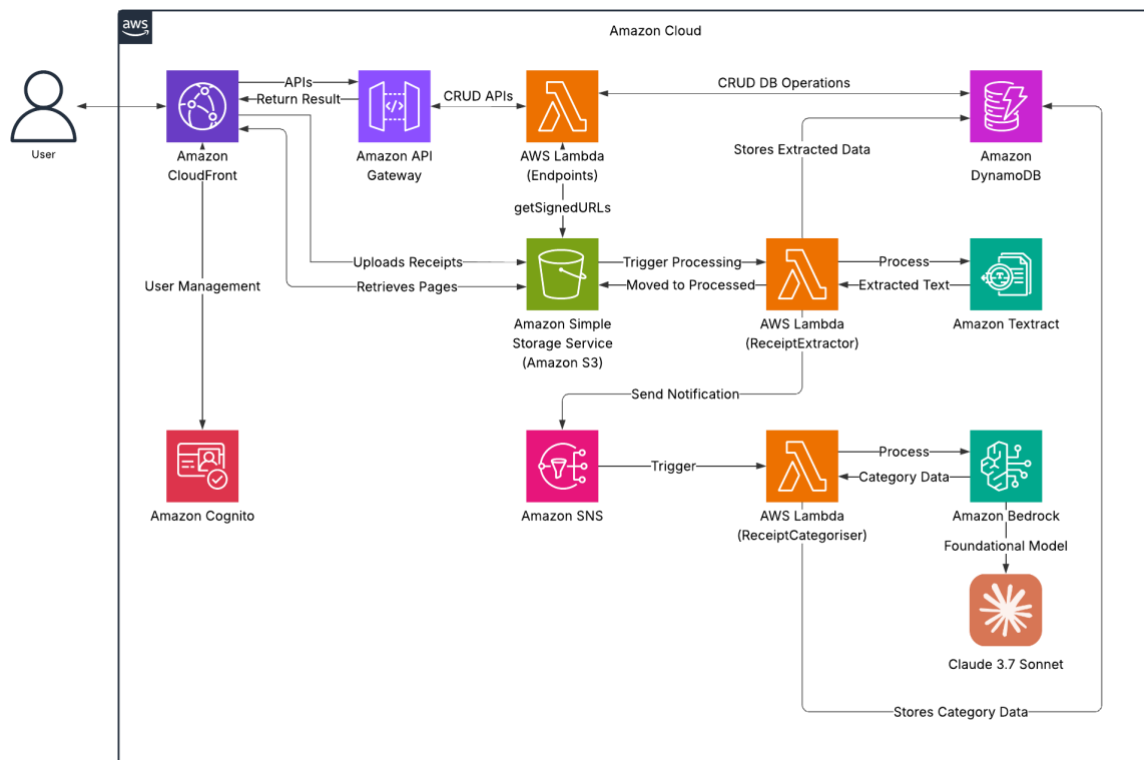


Figure 1 – System Architecture Diagram

5.2.1. User Interaction Layer

This layer represents the system's external interface, enabling users to interact with the application through the web interface and manage secure authentication and access control. It handles all requests originating from the browser and ensures they are properly authenticated before accessing backend resources.

- **Amazon CloudFront**

Acts as the Content Delivery Network (CDN), caching and serving static frontend assets (HTML, JavaScript, CSS) from Amazon S3 to provide low-latency access globally.

- **Amazon Cognito**

Newly integrated to manage user authentication and authorisation. Cognito issues secure tokens (ID, access, and refresh tokens) after a successful login, which are required to call protected API endpoints through the API Gateway. Each user's receipts and processed data are isolated by Cognito identity.

5.2.2. API and Application Logic Layer

This layer is responsible for handling all application logic and API operations, serving as the bridge between user-facing interactions and backend data processing. It validates requests, enforces authentication, and executes business rules through serverless compute functions.

- **Amazon API Gateway**

Serves as the entry point for all backend APIs, including CRUD operations and upload endpoints. It validates incoming requests using Cognito authorizers and routes them to AWS Lambda functions.

- **AWS Lambda (Business Logic Functions)**

Multiple Lambda functions handle specific parts of the workflow:

- Upload Handler: Receives receipt uploads, stores them in S3, and triggers the extraction process.
- CRUD Handler: Interfaces with DynamoDB to perform create, read, update, and delete operations for receipt and category data.
- Processing Handlers: Orchestrate text extraction (Textract) and categorisation (Bedrock) pipelines asynchronously.

5.2.3. Storage and Processing Layer

This layer is responsible for data persistence and for storing intermediate files. It ensures that both structured and unstructured data are securely stored and readily accessible for downstream AI processing and user retrieval.

- **Amazon Simple Storage Service (S3)**

Used for two main purposes:

- Hosting the static frontend (served through CloudFront).
- Storing uploaded receipt images, which act as triggers for downstream Lambda functions.

- **Amazon DynamoDB**

Provides serverless NoSQL data storage for all user-related data, including receipt metadata, extracted items, and categorised results. Each record is keyed by user identity (from Cognito) to ensure data segregation.

5.2.4. AI and Processing Pipeline

This layer serves as the system's intelligence core, leveraging AI services to extract and categorise receipt data. It integrates text recognition (Textract) and semantic analysis (Bedrock) through event-driven Lambda orchestration.

- **Amazon Textract**

Automatically extracts text content from uploaded receipt images. The extracted structured text (e.g., item names, quantities, prices) is passed to a Lambda function for further processing.

- **Amazon Bedrock**

Processes the structured text from Textract to perform semantic categorisation using a foundation model. Bedrock's output determines the expense category for each item, which is then stored in DynamoDB.

- **AWS Lambda (Pipeline Orchestration)**

Coordinates the data flow between Textract, Bedrock, and DynamoDB. Each step is invoked asynchronously via event triggers or notifications.

5.2.5. Event and Notification Layer

This layer supports asynchronous communication between distributed components, allowing independent execution of AI and backend processes while maintaining data flow continuity. It is implemented using Amazon Simple Notification Service (SNS), which orchestrates message passing between Lambda functions and other services. SNS topics are published after text extraction and categorisation to trigger subsequent processing stages and serve as the communication channel for user-facing updates, such as "receipt processed successfully."

5.2.6. Data Flow Summary

The following sequence outlines the end-to-end data flow within the ReceiptCAT system, describing how data moves across different AWS-managed layers from user interaction to AI processing and visualisation. This flow demonstrates how serverless components interoperate seamlessly to achieve automation, scalability, and real-time user feedback.

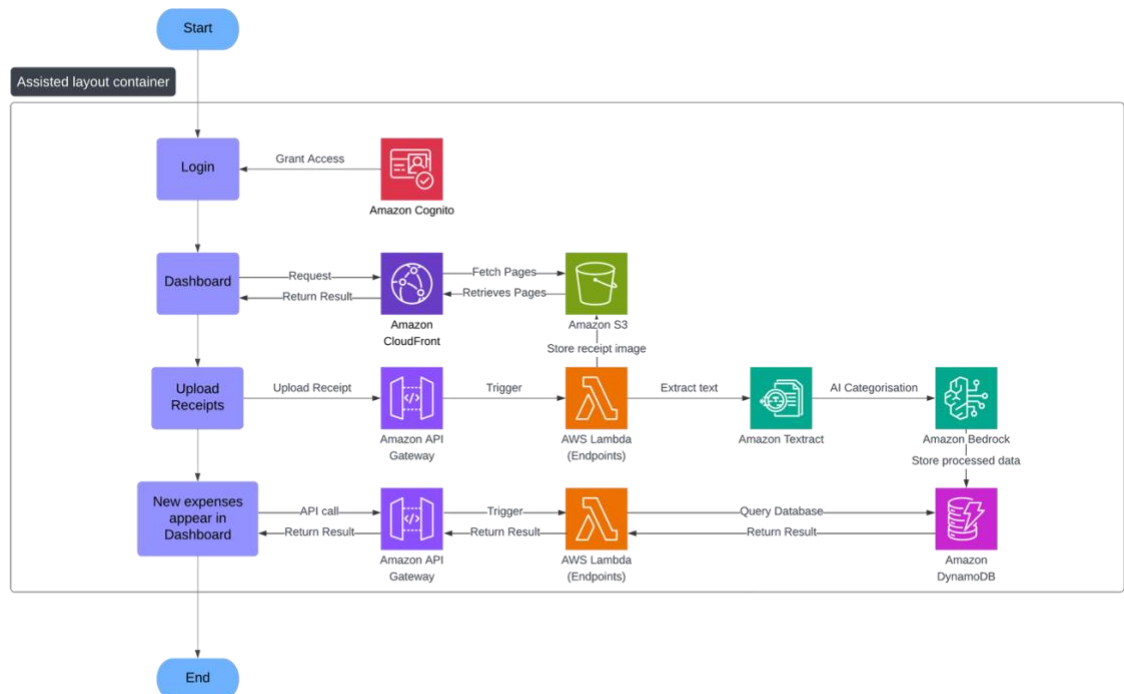


Figure 2 Simplified Data and User Flow chart

1. The user logs in through Amazon Cognito, gaining secure access to the application.
2. Through CloudFront, the user retrieves the frontend hosted in Amazon S3.
3. The user uploads a receipt via the API Gateway, which in turn invokes a Lambda function.
4. The uploaded image is stored in S3, triggering another Lambda that calls Amazon Textract.
5. Extracted text is sent to another Lambda, which calls Amazon Bedrock for categorisation.
6. Results are saved into DynamoDB, and a completion notification is sent via SNS.
7. The frontend makes API call to display the processed receipt and category summary.

5.2.7. Security and Access Control

System-wide security is implemented through AWS-managed identity, encryption, and network protection mechanisms to safeguard user data and enforce strict access boundaries.

The following components and configurations ensure comprehensive end-to-end protection:

- **Authentication and Authorisation:**

Managed by Amazon Cognito, integrating seamlessly with API Gateway.

- **Data Security:**

- S3 buckets are private, with access restricted by IAM roles and signed URLs.
- DynamoDB tables are partitioned by user identity to ensure isolation.

- **Network Security:**

- All communications use HTTPS through CloudFront and API Gateway endpoints.
- Least Privilege Principle: IAM roles and Lambda execution policies follow strict minimum access scopes.

5.2.8. Advantages of the Architecture

The ReceiptCAT architecture offers several operational and design advantages, leveraging AWS's managed services to deliver a robust, maintainable, and intelligent serverless solution.

- **Serverless Scalability:** No need to manage servers; scales automatically based on usage.
- **Cost Efficiency:** Pay-per-use model reduces idle costs.
- **Security Integration:** Cognito, IAM, and HTTPS ensure data privacy and compliance.
- **AI-Powered Insights:** Combines Textract (OCR) and Bedrock (LLM-based categorisation) for end-to-end intelligent automation.
- **Ease of Deployment:** Infrastructure managed via AWS CloudFormation and CI/CD pipeline for consistent delivery.

Each component's detailed functionalities, interactions, and responsibilities are further explained in [Section 5.3](#).

5.3. Functionalities/features

5.3.1. Functionality 1 – Secure Third Party and Email Login

This functionality provides users with a secure and seamless authentication experience using Amazon Cognito's Hosted UI, supporting both third-party social login (Google) and traditional email + password authentication. It ensures that only verified users can access their personal receipts and data within the ReceiptCAT system.

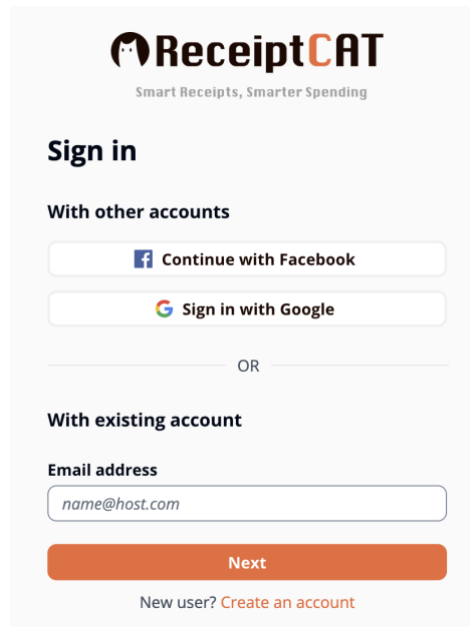


Figure 3 AWS Cognito Hosted Login

When users first access the web application via <https://d1iyjftsr6pf7t.cloudfront.net>, the frontend automatically checks for a valid Cognito authentication token (ID token or access token).

- If a valid token exists, the user is considered authenticated and can proceed to use the application normally.
- If no token is found (i.e., first-time visitor or expired session), the app redirects the user to the Cognito Hosted UI for authentication.

The Hosted UI presents two sign-in options:

- Continue with Google – authenticates via the user's Google account using OAuth 2.0.
- Email and Password Login – authenticates via Cognito's built-in user pool. Cognito sends a verification code to the user's email for confirmation before completing the signup or login process.

Upon successful authentication, Cognito issues three tokens (ID, Access, and Refresh) and redirects the user back to the application's redirect URI (the CloudFront distribution domain), with an authorisation code appended to the URL parameters.

- **Technical Workflow**

The technical flow follows the OpenID Connect (OIDC) and OAuth 2.0 Authorisation Code Grant standards. The key stages are outlined below:

1. Redirect to Cognito Hosted UI

When no valid token is detected in local storage, the frontend triggers a redirect to Cognito's Hosted UI endpoint, e.g.:

```
https://receiptcat.auth.ap-southeast-2.amazoncognito.com/login?
```

```
response_type=code
```

```
&client_id=<CLIENT_ID>
```

```
&redirect_uri=https://d1iyjtsr6pf7t.cloudfront.net/app
```

`response_type=code` indicates that the app requests an authorisation code.

The `redirect_uri` specifies where Cognito sends the user after login.

2. User Authentication via Cognito

Cognito authenticates the user using either:

- Google as an Identity Provider (IdP) through federated OIDC flow, or
- Cognito's User Pool for native email + password login with email verification.

Cognito then generates an authorisation code and redirects the user back to the specified redirect URI.

3. Authorisation Code Exchange

The frontend (or backend, depending on implementation) sends this code to Cognito's `/oauth2/token` endpoint to exchange it for tokens:

- ID Token: contains user identity claims (name, email, sub ID).
- Access Token: authorises API access via API Gateway.
- Refresh Token: used to obtain new tokens when the session expires.

4. Token Handling and Session Management

The frontend stores the tokens securely (typically in memory or encrypted local storage).

Each subsequent API request includes the Access Token in the Authorisation header as a Bearer token, allowing Amazon API Gateway to validate it using Cognito Authorizers.

When tokens expire, the app uses the Refresh Token to silently refresh the session without requiring the user to log in again.

5. User Context Resolution

Once validated, the user's Cognito Identity ID is propagated across backend services (e.g., DynamoDB partition key) to ensure strict data isolation between users.

- **Error Handling**

1. **Redirect Failure:** If the automatic redirect to the Cognito Hosted Login page fails (for example, due to network interruption or misconfigured redirect URIs), the `signinRedirect()` method catches the exception and logs a detailed error message to the browser console (`signinRedirect failed:`), preventing infinite redirect loops.
2. **Authentication Context Errors:** If the OIDC authentication context (`useAuth`) encounters an unexpected error (e.g., invalid session state or malformed token response), the SDK halts further redirection attempts until the page is refreshed, safeguarding against repeated login failures.
3. **Environment Misconfiguration:** If the configured redirect URI (`NEXT_PUBLIC_OIDC_REDIRECT_URI`) is missing or mismatched with the Cognito App Client settings, the user remains on the login page, and a console error is logged for quick diagnosis.
4. **Network or Token Retrieval Errors:** If network issues or token exchange failures occur during the OIDC handshake, the login flow may temporarily stall; refreshing the page automatically reinitiates the authentication process.

- **Security and Compliance**

1. **OIDC & OAuth 2.0 Compliance:** The flow strictly adheres to secure standards for delegated authorisation.
2. **Token Validation:** Tokens are verified against Cognito's public keys (JWKS endpoint) by AWS API Gateway.
3. **Least Privilege Principle:** Each authenticated user's access is limited to their own resources via identity-based policies.
4. **HTTPS-Only Communication:** All interactions between CloudFront, Cognito, and API Gateway are encrypted using TLS 1.2+.
5. **Email Verification Enforcement:** Prevents unverified or malicious email registrations.

- **Integration Summary**

1. **Frontend:** Performs token detection, redirection, and storage using the Cognito SDK and standard OIDC parameters.
2. **Cognito Hosted UI:** Manages user authentication and federated login via Google.
3. **API Gateway:** Validates tokens before granting API access.
4. **Backend (Lambda):** Extracts user identity from token claims (e.g., `sub` or `email`) to associate data records with the correct user.

5.3.2. Functionality 2 – Preview Editing

This functionality begins when a user clicks the Upload button from the navigation bar on any page and selects an image file to upload. Once a file is chosen, the selected image—along with its metadata—is temporarily stored in sessionStorage to maintain state consistency during navigation. The system then automatically redirects the user to the /app/preview page, where the stored file is retrieved and rendered for display and editing.

Using sessionStorage ensures that the selected file persists across navigation transitions without requiring re-selection. It also supports validation of file format and size, accepting only JPG, JPEG, and PNG files up to 5 MB.

On the preview page, users can review and make light edits to their receipt image before uploading. The editor provides four primary operations—Blackout, Crop, Reset, and Reselect Image—allowing users to remove sensitive information and focus on relevant areas of the receipt, thereby improving the accuracy of subsequent Amazon Textract and Amazon Bedrock processing. The interface for this feature is illustrated in *Figure 3* below.

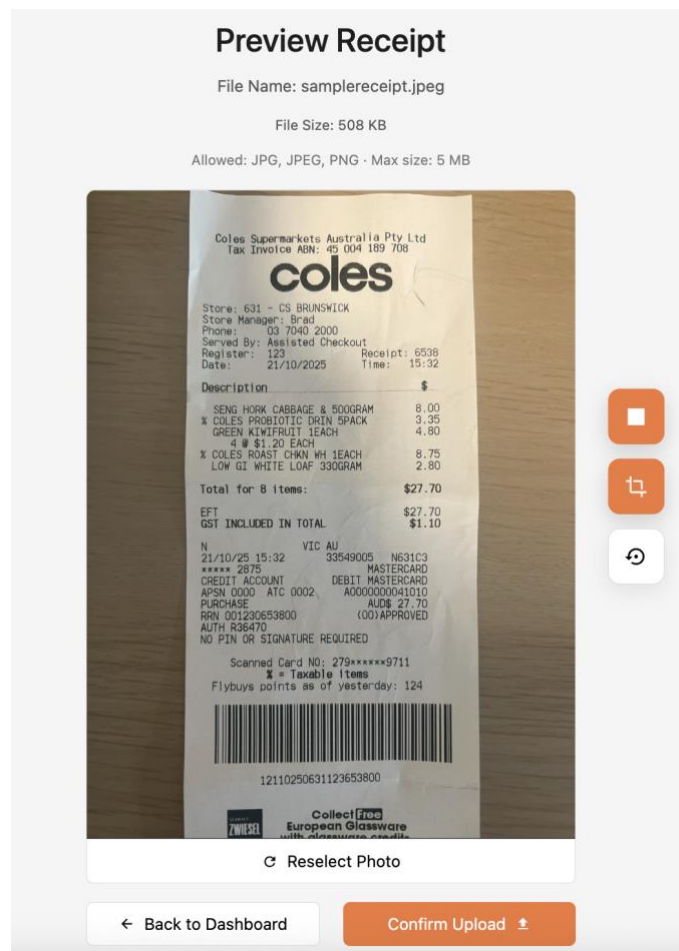


Figure 4 Preview & Editing UI

5.3.2.1. Feature 1 – Blackout Mode

This feature allows users to manually obscure sensitive information, such as card numbers, barcodes, or personal identifiers, in the uploaded receipt image. When activated, the interface overlays a drawing layer that lets users drag to define rectangular blackout areas. Each blackout region is rendered as a filled opaque block and recorded in the local editing state. The tool ensures that blackout operations are mutually exclusive with cropping actions—activating blackout mode automatically disables crop mode. On confirmation, the blackout layer is permanently merged into the image bitmap before upload, ensuring that unmasked data never leaves the client.

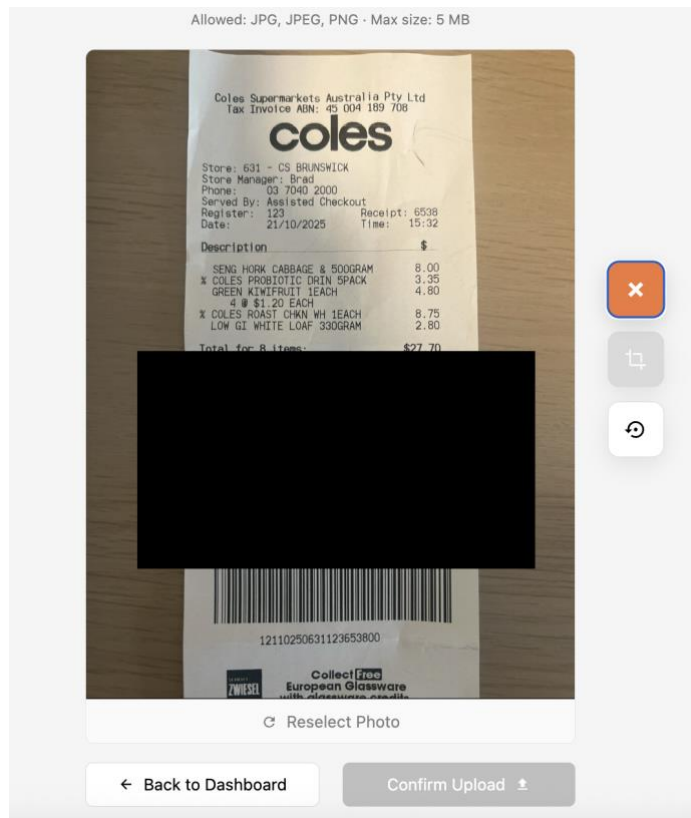


Figure 5 Blackout Mode

• Technical Workflow

1. When Blackout Mode is activated, the editor overlays an interactive layer on the image, allowing the user to define regions to obscure.
2. Toolbar state: Blackout is mutually exclusive with other edit tools. While Blackout Mode is on, Crop and all non-essential actions are disabled; only Reset remains enabled. Disabled controls are reflected in the UI.
3. On drag start, the system captures the starting position and creates a temporary selection rectangle constrained to the visible bounds.
4. As the user drags, the selection rectangle updates in real time to reflect the intended blackout region.
5. On drag end, the selection is finalised and added to the in-memory blackout set, which is rendered as opaque blocks over the preview (no persistent storage has been updated yet).

6. When the user confirms a blackout, each blackout rectangle is transformed from display coordinates to original-image pixels using the Cropper's scaling and offsets.
7. The system draws the original image at full resolution onto a new canvas and permanently fills the blackout regions with solid black (irreversible redaction).
8. The redacted image is encoded to the appropriate MIME (PNG or high-quality JPEG), converted to a new File, and becomes the current working copy.
9. The preview updates immediately to show the redacted version for a seamless UX.
10. At confirmation time, the system updates sessionStorage working entries (e.g., `fileUrl`, `fileInfo`, `fileType`) with the new object URL and metadata, while original entries remain preserved for potential reset. The new file is re-validated against type/size limits before saving.
11. If the user cancels instead of confirming, the system discards both draft and committed blackout shapes in memory and does not modify sessionStorage; the preview returns to the unedited state.

- **Error Handling**

1. Ignores invalid blackout attempts such as zero-area rectangles or gestures outside the overlay region.
2. Handles cases where Cropper data or the image element is unavailable by returning early from the bake function.
3. If canvas creation or drawing fails, it triggers the `onError` callback with a descriptive message.
4. Converts export failures (e.g., null blob or invalid data URL) into non-fatal `onError` notifications.
5. Gracefully resets internal states (`draftRect`, blackouts, drag anchors) after any failure to maintain consistent UI behaviour.

5.3.2.2. Feature 2 – Crop Mode

This feature allows users to trim unnecessary or irrelevant parts of a receipt image, keeping only the section that contains meaningful information for analysis. When activated, the interface enables a draggable and resizable crop box that lets users precisely select the desired area. Cropping is mutually exclusive with blackout and other tools—activating crop mode automatically disables them. Once confirmed, the selected region is extracted and saved as a new working image, ensuring that only relevant content proceeds to AI processing.

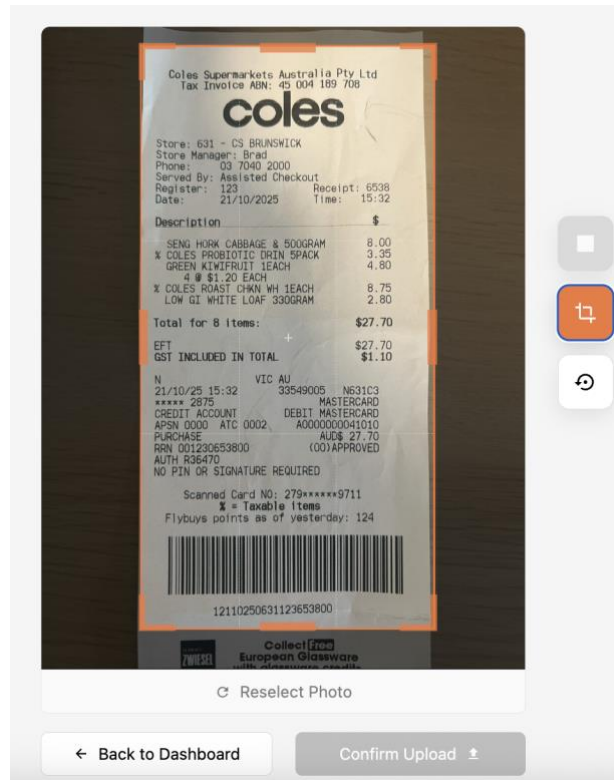


Figure 6 Crop Mode

• Technical Workflow

1. When Crop Mode is activated, the editor enables a draggable/resizable crop box on top of the image and disables other non-essential tools (Crop is mutually exclusive with Blackout and other actions; Reset remains available).
2. As the user drags the handles or moves the crop box, the selection updates in real time, providing precise visual feedback of the retained region.
3. The system enforces basic validity constraints on the crop region (e.g., minimum width/height) and clamps movement within the visible image bounds.
4. On Confirm Crop, the editor retrieves the current crop metrics from the Cropper (image-space data and canvas-space cropBox), then generates a high-quality cropped canvas (imageSmoothingEnabled = true, imageSmoothingQuality = "high").
5. The cropped canvas is encoded with the appropriate MIME type (PNG or high-quality JPEG, depending on the current file name) and converted to a new File, which becomes the current working copy.
6. The preview updates to reflect the cropped result (either by replacing the displayed image or by reloading the newly created working file), ensuring a seamless visual transition out of Crop Mode.

7. At this confirmation point, the system updates sessionStorage working entries (e.g., `fileUrl`, `fileInfo`, `fileType`) with the cropped file's object URL and metadata. In contrast, original entries remain preserved to support a full Reset. File constraints (type: JPG/JPEG/PNG; size ≤ 5 MB) are re-validated against the cropped output before saving.
8. The editor lifts the latest crop state (e.g., via `onCropStateChange`) so the selection can be restored after navigation or a soft reload, if the caller desires.
9. Exiting Crop Mode without confirmation (e.g., cancel) discards the transient crop box and does not modify sessionStorage; the working/original files remain unchanged.

- **Error Handling**

1. No selection/too small selection: If the crop box is missing or below the minimum dimensions, confirmation is treated as a safe no-op (nothing is exported or persisted).
2. Unavailable Cropper/image element: If Cropper or its image metrics are unavailable, the operation aborts early without side effects.
3. Canvas or encoding failure: If the cropped canvas cannot be created or `toBlob` returns null, the system surfaces a descriptive `onError` and keeps the current working/original entries intact.
4. File constraint violations: If the cropped file's type is unsupported or its size exceeds 5 MB, the save is rejected, a clear `onError` is emitted, and no sessionStorage write occurs.
5. Storage issues: If writing to sessionStorage fails (quota/unavailable), the system reports a user-friendly error and retains the previous working/original URLs; any newly created object URLs that were not committed are revoked.
6. State cleanup: After any failure or cancel, transient UI state (crop box, drag state) is cleared, and the toolbar returns to a consistent non-conflicting state (other tools remain disabled only while Crop Mode is active; Reset stays available).

5.3.2.3. Feature 3 – Reset to Origin

This feature allows users to revert the preview image to its original, unedited version. It retrieves the initially uploaded file and metadata from sessionStorage and reloads them into the preview interface. This ensures that any applied edits—such as blackouts or crops—can be safely discarded, restoring the receipt image to the same state it was in immediately after selection.

- **Technical Workflow**

1. When the Reset button is clicked, the system calls the `resetToOriginal()` method provided by the file selection module.
2. This function retrieves the original upload entries from sessionStorage, including `origFile`, `origFileUrl`, `origFileInfo`, and `origFileType`.
3. The system verifies that these original entries exist and are still valid.
4. If valid, the system restores these values as the active working entries (`fileUrl`, `fileInfo`, `fileType`), replacing any previously edited or cropped versions.
5. Once the working entries are updated, the controller invokes `loadFromSession()` to refresh the preview view, reloading the unedited receipt image into the Cropper.
6. All active editing states—such as Blackout, Crop, or temporary working files—are cleared, and the editor returns to its default idle state.

- **Error Handling**

1. Missing original data: If the original sessionStorage entries (origFile or origFileUrl) are missing or corrupted, the system aborts the reset process and displays an error message (e.g., “Unable to restore original image — source file missing.”).
2. Invalid or revoked file reference: If the stored original URL is no longer valid, the system clears the invalid entries and prompts the user to re-upload the receipt.
3. Preview reload failure: If the original file cannot be reloaded into the Cropper (e.g., unsupported MIME type or URL access issue), the system raises an onError notification while retaining the last working preview to prevent a blank view.
4. sessionStorage write failure: If re-synchronising working entries to sessionStorage fails (e.g., due to quota or browser limitations), the system logs the issue, preserves the previous working state, and notifies the user.
5. State cleanup after failure: Regardless of the error type, transient edit states (Blackout or Crop) are cleared to maintain consistent tool behaviour and avoid residual UI conflicts.

5.3.2.4. Feature 4 – Reselect Photo

This feature lets users replace the current working image with a newly selected file directly from the preview screen. It uses a hidden file input bound to the Reselect Photo button, preserves the original (unmodified) upload for full reset, and updates the working copy only after validating type and size constraints.

- **Technical Workflow**

1. When the user clicks Reselect Photo, the UI programmatically triggers the hidden `<input type="file">` element to open the native file picker (accepts image/png, .jpg, .jpeg).
2. If the user selects a file, the handler (onReselectFile) reads the file and performs client-side validation against the configured constraints (JPG/JPEG/PNG, ≤ 5 MB).
3. On successful validation, the system creates a new object URL for the selected file and updates the working entries in sessionStorage (e.g., fileUrl, fileInfo, fileType, orig*).
4. Any active editing modes (Blackout/Crop) are deactivated, and their transient state is cleared to avoid conflicts with the newly selected image.
5. The preview is refreshed (e.g., via loadFromSession() in the controller) so the new working image is rendered immediately in the Cropper/editor.
6. The previous working object URL (if any) is revoked to prevent memory leaks.
7. While reselection or follow-up processing is in progress—or when an edit mode requires exclusivity—the Reselect Photo control is disabled and aria-disabled is set for accessibility.

- **Error Handling**

1. No selection/user cancels: If the file picker is closed without a selection, the operation is treated as a no-op; no state changes occur.
2. Unsupported type or oversize file: If validation fails (non-JPG/PNG or size > 5 MB), the system surfaces a clear error and does not update sessionStorage; the previous working image remains in place.
3. Object URL or read failure: If creating an object URL or reading the file fails, an onError message is shown, and the prior working/original entries are left intact.
4. sessionStorage write failure: If updating working entries cannot be persisted (quota/unavailable), the system reports a user-friendly error and rolls back to the previous working image; any newly created object URL is revoked.
5. Preview reload failure: If the new file cannot be rendered in the Cropper (e.g., invalid image data), the system emits an onError event, restores the previous working image, and clears transient edit state to maintain a consistent UI.
6. Tool-state conflicts: Attempts to reselect while a mutually exclusive tool is active are blocked; the button remains disabled (and announced via aria-disabled) until the conflicting state is cleared.

5.3.2.5. Feature 5 – Confirm Upload

This feature finalises the editing flow by securely uploading the current **working image** to the backend using an authenticated request. It validates authentication, delegates the upload to the presigned-URL flow, and coordinates UI state (labels, routing, error visibility) during and after the operation.

- **Technical Workflow**

1. When the Confirm Upload button is clicked, the system calls the onConfirm() method from the preview controller.
2. The controller retrieves the user's authentication token, preferring the OIDC id_token from Cognito (via react-oidc-context) and falling back to the custom accessToken hook if necessary.
3. With a valid token, the controller triggers the uploadWithToken() function from the usePresignedUpload hook.
4. This function sends a request to the backend /presign endpoint to obtain a presigned S3 upload URL, which is authorised using the user's token. Refer to [Section 5.3.3](#) for more details on acquiring and uploading the presigned URL.
5. Once the presigned URL is received, the client uploads the current working image file directly to Amazon S3 using an HTTPS PUT request.
6. During the upload process, all other user interactions and editing controls are disabled to prevent interruption or inconsistent state changes.
7. After a successful upload, the session storage entries are cleared, and the system marks the upload as complete. A confirmation toast message is displayed to the user.
8. The application then automatically redirects the user to the Receipt History page (/app/history), where the newly uploaded receipt appears in their record list.

• Error Handling

1. Unauthenticated user: If neither an OIDC id_token nor an accessToken is available, the system immediately aborts the upload and displays an alert message (“*Not authenticated. Please sign in again.*”). No upload request is sent to the backend.
2. Presigned URL or upload failure: If an error occurs while requesting the /presign endpoint or uploading the file to Amazon S3, the system sets the internal error state through the usePresignedUpload hook. The upload button label changes to “Retry Upload,” allowing the user to retry the operation.
3. Network or timeout error: If network connectivity is lost or the upload times out, the system reports the failure through the error field in the controller. The interface re-enables the Confirm Upload button after the failure, while ensuring that all other interactions are disabled during the retry process.
4. Unauthorised or expired session: If the backend rejects the token (e.g., due to expired or invalid credentials), the upload fails and the user is prompted to re-authenticate before retrying.
5. UI consistency and routing suppression: To prevent transient error pop-ups during navigation after a successful upload, the controller sets isRouting = true. This ensures a smooth redirect flow without flashing stale error messages.
6. Session cleanup on failure: If an upload fails, sessionStorage data remains intact (the working file and metadata are preserved), so the user can safely retry the upload without losing progress.

5.3.3. Functionality 3 – Upload Receipt

This functionality enables authenticated users to upload receipt images into the system. The design ensures scalability, security, and data ownership isolation by leveraging AWS Cognito, API Gateway, Lambda, and Amazon S3.

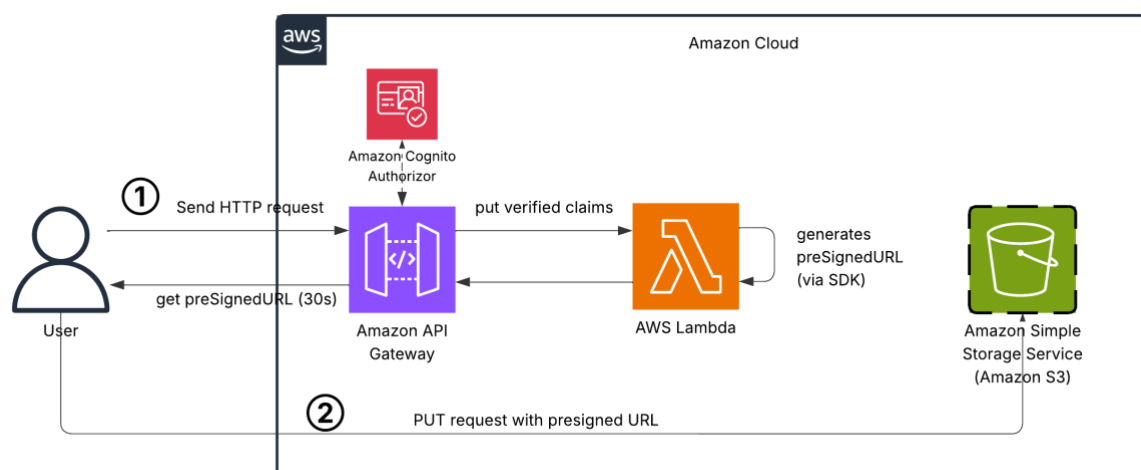


Figure 7 Upload Receipt Flow

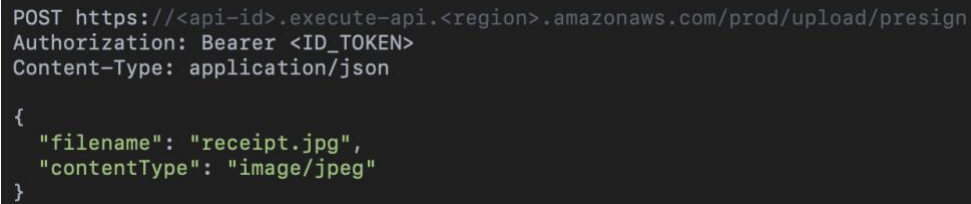
- **Technical Workflow:**

1. User Authentication:

A logged-in user will have a valid ID token (JWT) returned from Cognito and return it to the frontend application. (Login through Cognito details, please refer to [Section 5.3.1](#))

2. Request for Upload URL:

Once the user clicks the “Confirm Upload” button at the preview page, the frontend issues an HTTP POST request to the API Gateway endpoint:



```
POST https://<api-id>.execute-api.<region>.amazonaws.com/prod/upload/presign
Authorization: Bearer <ID_TOKEN>
Content-Type: application/json

{
  "filename": "receipt.jpg",
  "contentType": "image/jpeg"
}
```

Figure 8 POST Request example snippet

The API Gateway is configured with a Cognito User Pool Authorizer (details are available in our CDK stack).

The authorizer validates the ID token before forwarding the request to Lambda. If validation fails (e.g., token missing, expired, or invalid), API Gateway immediately returns 401 Unauthorised or 403 Forbidden without invoking Lambda. If validation succeeds, the request proceeds to the presign-upload Lambda function with the verified claims.

3. Presigned URL Generation (Lambda)

The Lambda function extracts the sub (user unique identifier) from the JWT claims from API Gateway. It generates a unique S3 object key under the user’s namespace: tmp/users/<sub>/<timestamp>_<uuid>_<sanitizedFilename>. A corresponding presigned PUT URL is generated using the AWS SDK with a 30-second expiry.

4. Direct Upload to S3

The presigned URL is then returned in the Lambda response and passed back through API Gateway to the frontend. Using this URL, the frontend uploads the receipt image directly to Amazon S3 via an HTTPS PUT request. This design ensures that binary file traffic bypasses API Gateway and Lambda entirely, significantly improving upload performance and reducing operational costs. The frontend, therefore, relies solely on Lambda for the presigned URL and does not route the actual file contents through the API layer.

- Error Handling

1. Missing or Invalid Token: If the user's Cognito authentication token is missing, expired, or malformed, the presign request is rejected with a 401 Unauthorised response, preventing unauthenticated uploads to S3.
2. Presign URL Generation Failure: If the /presign Lambda fails to generate a valid URL (for example, due to permission misconfiguration, AWS SDK errors, or missing environment variables), the client catches the error, logs it to the console, and halts the upload process.
3. Expired Presigned URL: Each presigned URL is valid for only 30 seconds. If the upload begins after expiration, S3 returns a 403 Forbidden error, prompting the user to retry and obtain a new URL.
4. Network or Upload Failure: Network interruptions or failed HTTPS PUT requests to S3 trigger the onError callback in the client, displaying an error toast and preventing incomplete files from being stored.
5. File Validation Errors: Before initiating the upload, the client validates file type and size. Files exceeding 5 MB or with unsupported extensions (.jpg, .jpeg, .png only) are rejected immediately, with an inline error message displayed.
6. Post-Upload Synchronisation Failure: If the upload completes but the subsequent metadata sync (e.g., updating session storage or redirecting to /app/history) fails, the system logs the issue locally while retaining the successfully uploaded file in S3 for recovery.

- Security and Data Isolation

Uploaded files are stored in S3 under a user-specific namespace (tmp/users/<sub>/<timestamp>_<uuid>_<sanitizedFilename>, S3 storage design please refer to [Section 5.4](#)). This ensures that each user's data is logically isolated and prevents unauthorised access between users. Bucket policies and IAM configurations enforce strict access controls, ensuring that presigned URLs can be used only for the intended object within their limited lifetime (30 seconds). All communication between the frontend and S3 is secured via HTTPS, and S3 enforces server-side encryption to protect data at rest. Combined with Cognito-based authentication and authorisation, this guarantees both secure file transfer and multi-tenant data isolation.

5.3.4. Functionality 4 – Receipt Text Extraction

For automated text extraction, the system employs Amazon Textract's AnalyzeExpense API, a pre-trained, domain-optimised model for parsing financial documents such as invoices and receipts.

Unlike Textract's general-purpose OCR endpoints—DetectDocumentText (raw text detection) or AnalyzeDocument (form and table analysis)—AnalyzeExpense applies an AWS-trained machine learning model specifically designed to identify structured financial entities such as `VENDOR_NAME`, `INVOICE_RECEIPT_DATE`, `TOTAL`, and `LINE_ITEM_GROUPS`.

- **Textract Model Selection – AnalyzeExpense**

For the extraction of receipt data, the system leverages Amazon Textract's AnalyzeExpense API, a specialised deep learning model designed to parse semi-structured financial documents such as invoices and receipts.

Unlike the generic DetectDocumentText or AnalyzeDocument APIs, which only return raw OCR text or basic key-value relationships, AnalyzeExpense performs domain-specific semantic extraction — it recognises business fields such as `VENDOR_NAME`, `INVOICE_RECEIPT_DATE`, `TOTAL`, and detailed `LINE_ITEM_GROUPS`.

This model was chosen for several reasons:

1. It provides semantic understanding of expense-related documents out of the box, enabling accurate detection of totals, vendors, and line items without custom field mapping.
2. It significantly reduces post-processing complexity, as Textract already groups related key-value pairs and item lists into a unified expense schema.
3. It ensures consistent field naming across diverse receipt templates, improving reliability for downstream data categorisation and DynamoDB storage.
4. Because the model is pre-trained and fully managed by AWS, it eliminates the need for custom dataset collection, annotation, and ML model training—activities that were not feasible within the project's limited development timeframe.

Other Textract models, such as DetectDocumentText and AnalyzeDocument, were evaluated but deemed less suitable, as they require extensive rule-based parsing and custom ML training to reach comparable accuracy.

By leveraging AnalyzeExpense, the project achieves a strong balance between accuracy, maintainability, and implementation efficiency, aligning with the project's resource and time constraints.

Further details on the Textract evaluation and analysis can be found in our research report — [P000340SE-EvaluationReport-Textract](#).

- **Technical Workflow**

1. Event Trigger (S3 → Lambda): An object-created event under tmp/users/{sub}/receipts/ invokes the Receipt Extractor Lambda with the S3 bucket and key.
2. Key Validation & Ownership Derivation: The Lambda validates the object key format (timestamp + UUID + sanitised filename) and derives the Cognito sub (user identifier) from the path to ensure tenant isolation.
3. Object Existence Guard: A HeadObject check verifies that the target S3 object exists and is readable before processing.
4. Textract – AnalyzeExpense: The Lambda calls Textract AnalyzeExpense on the S3 object to obtain structured fields and line items.
5. Field Normalisation:
 - Date: Parsed and normalised to YYYY-MM-DD (e.g., via a deterministic date parser) to standardise storage.
 - Amounts: Currency strings are sanitised (symbols/commas removed, decimal handling) and rounded to 2 decimal places.
 - Vendor: Whitespace is trimmed; empty values default to "Unknown".
6. Line-Item Construction: Each detected line item is mapped into an object { name, price, quantity } with numeric coercion and defaults (quantity = 1, price = 0.00 when missing).
7. Total Fallback Guard: If Textract returns a total of 0, the system sums line-item prices as a fallback, rounding to 2 decimal places to ensure a reasonable aggregate is always present.
8. S3 Path Finalisation: The processed object's final path is computed by removing the tmp/ prefix (e.g., tmp/users/... → users/...), and this final S3 URI is attached to the receipt record.
9. DynamoDB Upsert: A new item is inserted into the Receipts table with the (user_id, receipt_id) key and the following attributes: date, vendor, total, items (JSON string), s3_path, and processed_at (timestamp).
10. Publish Notification (SNS): A compact JSON of the processed receipt is published to an SNS topic to fan out events (e.g., for categorisation or analytics).
11. S3 Move (Copy + Delete): The object is copied from tmp/... to users/... (permanent storage) and the original tmp object is deleted to complete the hand-off from staging to long-term storage.

- **Error Handling**

1. Invalid Key Structure: If the S3 key does not match the expected pattern (tmp/users/{sub}/receipts/{timestamp}_{uuid}_{safeName}), processing is aborted to prevent malformed or cross-tenant inputs.
2. Missing Object: If HeadObject fails, the Lambda stops with an explicit error (object not found), avoiding unnecessary Textract calls.
3. Textract Failures: Exceptions from AnalyzeExpense propagate as processing errors; no DynamoDB write or S3 moves occur on failure.
4. Normalisation Issues: Unparseable dates default to “today” in YYYY-MM-DD format; non-numeric amounts default to 0.00, with warnings recorded in logs.
5. DynamoDB Write Errors: Put failures terminate the flow; the object remains in tmp/ and no SNS message is emitted, ensuring downstream consumers do not see partial data.
6. S3 Move Failures: If copy or delete fails, the error is surfaced and logged; the DynamoDB item already contains the final s3_path, so an operator can retry the move (or a compensating task can reconcile tmp/ remnants).

5.3.5. Functionality 5 – AI Categorisation

This functionality performs AI-based categorisation of extracted receipt data using Amazon Bedrock and the Anthropic Claude model.

After the Textract stage completes, a structured receipt object is published to an SNS topic, which triggers the Receipt Categoriser Lambda. This Lambda formats the receipt details into a text prompt and sends it to the Claude model for semantic classification of each line item into pre-defined expense categories. The categorisation results are then normalised and written back to the corresponding DynamoDB record, enriching each item with a standardised category attribute.

Further details on the Bedrock evaluation and analysis — including model accuracy benchmarking, pricing estimation, and the design of the predefined category taxonomy — can be found in our supplementary research report P000340SE-EvaluationReport-Bedrock.

- **Technical Workflow**

1. **Event Trigger (SNS → Lambda):** When the Textract Extractor Lambda publishes a processed receipt to the SNS topic, the Categoriser Lambda is automatically invoked with the receipt payload containing vendor, date, total, and item details.
2. **Prompt Construction:** The Lambda formats the receipt data into a structured text block wrapped in `<receipt>` tags, combining it with the predefined system prompt (`SYSTEM_PROMPT_NO_SHOT`) to form the model input.
3. **Model Invocation:** The formatted prompt is sent to Amazon Bedrock, which invokes Anthropic Claude 3.7 Sonnet via the `InvokeModelCommand`. The request uses a JSON payload specifying the Bedrock API version (`bedrock-2023-05-31`) and a token limit of 2048.
4. **Response Parsing:** The model's response body is decoded and analysed to extract predicted category names. The Lambda first attempts to parse the result as a JSON array; if unsuccessful, it falls back to splitting comma-separated text.
5. **Category Mapping:** Each category label returned by the model (e.g., *"Frozen"*, *"Health_Medicine"*) is mapped to a corresponding internal enumeration value used throughout the system (e.g., `frozen`, `health_medicine`, `other`).
6. **DynamoDB Update:** The Lambda retrieves the target receipt record via its (`user_id`, `receipt_id`) keys, updates each item object with its derived category value, and writes the modified items array back to DynamoDB as a JSON string.
7. **Completion:** Once the record is updated, the categorised receipt becomes available for user display and dashboard analytics. The entire flow—from SNS event to DynamoDB update—operates asynchronously and requires no manual intervention.

- **Error Handling**

1. **SNS Message Parsing Failure:** If the incoming message cannot be parsed or lacks required receipt attributes, the Lambda aborts execution and logs an explicit parsing error.
2. **Model Invocation Errors:** Network, Bedrock throttling, or payload validation issues cause the Lambda to terminate gracefully without performing any DynamoDB updates.
3. **Unstructured Model Response:** Non-JSON or unparseable outputs trigger a fallback that defaults all items to `"Other"`.
4. **DynamoDB Operation Failure:** Read or write failures are logged in CloudWatch, and no partial updates are applied to avoid inconsistent states.
5. **Category Count Mismatch:** If the number of categories returned does not match the number of items, extra items are assigned `"Other"` to maintain array integrity.

5.3.6. Functionality 6 – Visualisation of Expenses

This functionality presents a visual summary of the user's spending through an interactive pie chart on the dashboard page. The chart helps users understand how their expenses are distributed across categories (e.g., *Frozen*, *Health & Medicine*, *Drinks*) within the selected date range.

By default, the dashboard displays data for the current month, but when the user adjusts the date selector, the system retrieves new data from the backend and updates the chart instantly to reflect the chosen period.

Users can also interact directly with the chart legend — clicking a category toggles whether it is included in the visualisation. This allows users to freely exclude or include specific categories from the calculation, enabling focused analysis on selected expense groups.

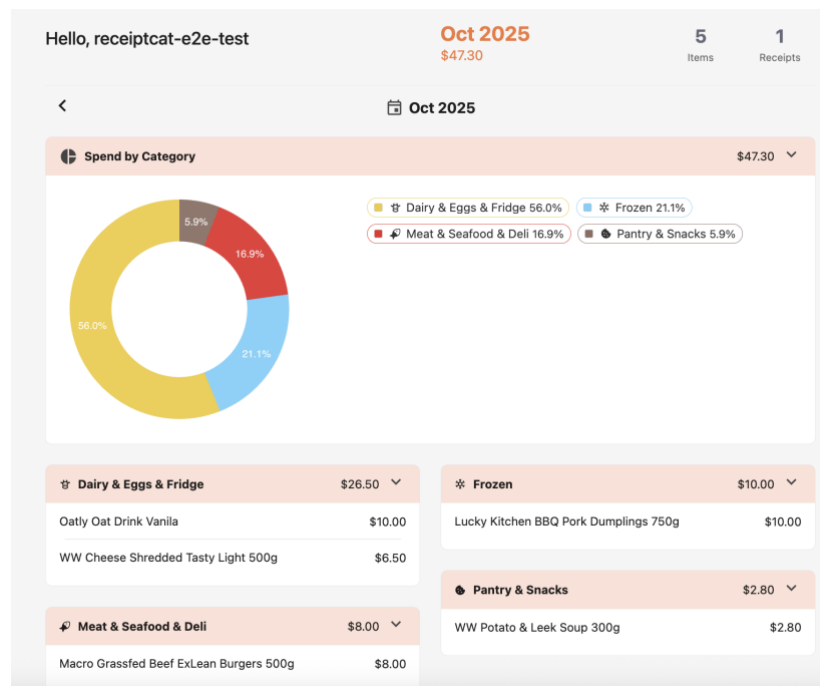


Figure 9 Dashboard Visualisation of Expenses

- **Technical Workflow**

1. **Data Retrieval from Backend:** When the dashboard page loads (or when the user changes the date range), the system sends a request to the backend API to retrieve all processed receipts for the authenticated user within that date range.
2. **Backend Aggregation:** The backend aggregates the receipts into category groups, calculating each category's total spending and the overall number of receipts. The API then returns a structured response containing all category names, totals, and line items.
3. **Frontend Data Processing:** After receiving the data, the dashboard processes it to calculate topline statistics (such as total spending and total receipts) and sort the categories by total amount in descending order. The processed data is then prepared in a format suitable for visual rendering.
4. **Chart Rendering:** Using the API response for the selected date range, the dashboard aggregates totals per category and computes each category's percentage of overall spend; this dataset is passed to the CategoryPieCard component, which renders a responsive pie chart with slice sizes proportional to those percentages and assigns colours by mapping each category key to a predefined colour palette.
5. **Automatic Updates:** When users navigate between months or manually select a different date range, the dashboard automatically sends a new query to the backend and re-renders the pie chart and related components to keep the displayed analytics consistent with the selected timeframe.

- **Error Handling**

1. **CORS Preflight Requests:** If the request is OPTIONS, the endpoint returns 200 OK with CORS headers and an empty body to satisfy browser preflight requirements.
2. **Missing Parameters:** If any of the `userId`, `fromDate`, or `toDate` is absent, the backend returns a 400 Bad Request with `{ error: "Missing parameters" }`.
3. **DynamoDB Query Failure:** If the DynamoDB query throws an exception (e.g., service unavailability, permission issues, or malformed query expressions), the handler logs the error and returns a 500 Internal Server Error with a generic message to prevent information leakage.
4. **Malformed Item Payload:** If parsing the `items` attribute (`JSON.parse(item.items?.S ?? "[]")`) fails due to invalid data, the same try/catch block catches the exception, and a 500 Internal Server Error is returned.
5. **Empty Result Set:** If no receipts match the specified date range, the endpoint still returns 200 OK with `{ receiptsCount: 0, categoryGroups: [] }`, allowing the frontend to render an empty state instead of an error.
6. **Query or Network Failure:** If a network issue or backend error occurs during data retrieval, the dashboard displays a red inline error message and retains the previously loaded results.
7. **Empty Data (No Receipts):** When the API response contains no data, the dashboard loads normally but shows an empty chart area with an explanatory message.
8. **Loading State:** While data is being fetched, a "Loading..." indicator is displayed to maintain user feedback and prevent chart flickering.
9. **All Categories Hidden (Legend Filter):** If the user hides all categories via the chart legend, the chart displays a message ("All categories are hidden. Use the legend to show some categories.") along with a Reset option to restore visibility.
10. **Client-Only Rendering Guard:** The pie chart component is dynamically imported with SSR disabled, ensuring it renders only in the browser and avoiding server-side rendering errors.

11. Safe Data Normalisation: All category totals received from the backend are normalised to numeric values; non-numeric or null inputs are coerced to defaults to prevent NaN or undefined values from breaking the chart.
12. Layout Measurement Fallbacks: If the chart container size cannot be determined immediately, the component falls back to a predefined default size range, preventing layout thrashing or rendering failures.
13. Unknown or Missing Category Keys: Any items with missing or unrecognised category keys are safely ignored during chart rendering, preserving the rest of the chart's data integrity and preventing runtime exceptions.

- **Data Requirements**

The visualisation feature relies on aggregated category-level expense data retrieved from the backend endpoint:

GET /users/{userId}/dashboard?fromDate=YYYY-MM-DD&toDate=YYYY-MM-DD.

The data returned by this endpoint serves as the single source of truth for both the summary statistics and the pie-chart visualisation.

1. Required Inputs:

- `userId` (path parameter, string): The Cognito sub value uniquely identifying the authenticated user.
- `fromDate` (query parameter, string): Inclusive start date for aggregation, in ISO format (YYYY-MM-DD).
- `toDate` (query parameter, string): Inclusive end date for aggregation, in ISO format (YYYY-MM-DD).
- `Authorisation` (header, string): The Cognito `id_token`, passed as a Bearer token for request authentication.
- If `fromDate` and `toDate` are omitted, the system automatically applies the current month as the default range.

2. Expected Backend Response Structure:

The backend aggregates all receipts for the specified period and returns per-category summaries in the following shape:

```
{
  "receiptsCount": 6,
  "categoryGroups": [
    {
      "category": "groceries",
      "items": [
        { "name": "Aldi Whole Milk 2L", "price": 3.5, "quantity": 1 },
        { "name": "Bread Loaf", "price": 2.0, "quantity": 1 }
      ],
      "total": 5.5
    },
    {
      "category": "drinks",
      "items": [
        { "name": "Pepsi Max 1.25L", "price": 3.5, "quantity": 1 }
      ],
      "total": 3.5
    }
  ]
}
```

- receiptsCount (number): Number of receipts processed within the date range.
- categoryGroups (array): Array of category-level summaries containing:
 - (1) category (string): Raw category key generated by the backend (e.g., "groceries", "drinks").
 - (2) items (array): Line items belonging to that category, including name, price, and optional quantity.
 - (3) total (number): Total spending amount for the category. If missing, the frontend derives it from the sum of item.price.

3. Frontend Visualisation Requirements:

- The pie chart requires each entry to contain:
 - (1) categoryKey: Canonical category identifier for colour mapping.
 - (2) category: Display label (mapped from CATEGORY_LABELS).
 - (3) total: Numeric total used to calculate slice proportions.
- The chart maps every categoryKey to a predefined colour palette to maintain visual consistency across sessions.
- The legend toggle modifies inclusion of categories in the displayed dataset; excluded categories are ignored in percentage calculations, while totals dynamically adjust.
- If all categories are hidden, the component displays an empty-state message prompting the user to reset visibility.

4. Backend Expectations:

- Must validate all required parameters (userId, fromDate, toDate) and reject invalid requests with 400 Bad Request.
- Queries DynamoDB using the user's partition key and date range, aggregating totals server-side.
- Returns 200 OK with an empty categoryGroups array and receiptsCount = 0 when no receipts are found.
- Provides valid CORS headers to enable secure browser access from the dashboard.

5.3.7. Functionality 7 – Time Filtering

This functionality allows users to filter dashboard analytics by a specific time range, enabling them to view spending summaries and category distributions for a selected period.

By default, the dashboard displays statistics for the current calendar month, providing an immediate overview of recent spending activity. Users can adjust this view by interacting with the date range selector located in the top-middle section of the dashboard.

When a new start and end date are selected, the system automatically re-fetches the corresponding receipt data. It updates all visual components — including total spending, category pie charts, and receipt counts — to reflect the chosen period.

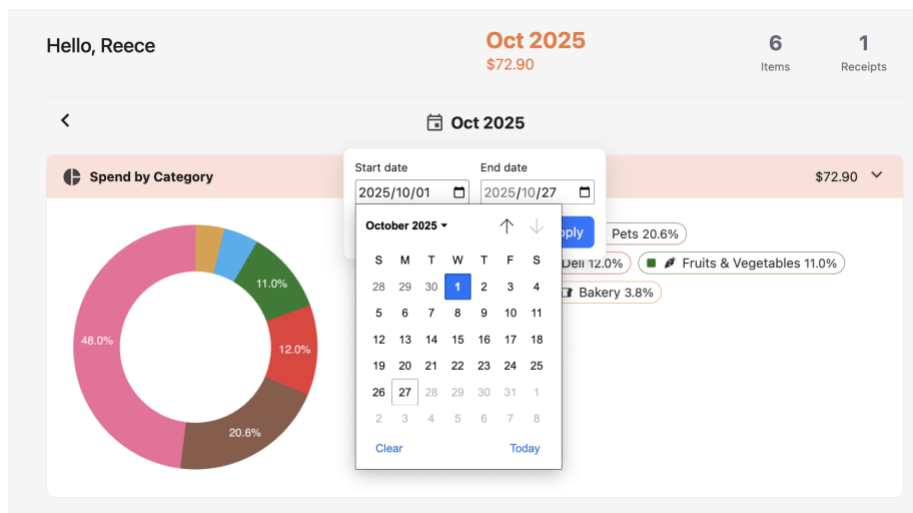


Figure 10 Dashboard Date Selector

• Technical Workflow

1. **Default State (Current Month):** Upon navigating to the dashboard, the front-end controller (useDashboard) automatically determines the current month's start and end dates and requests aggregated data for that range from the backend API.
2. **User Interaction (Date Selector):** When the user selects a new time range through the date picker UI, the component updates the internal state and triggers a refresh event.
3. **Backend Query:** The dashboard API receives the updated date parameters (e.g., startDate, endDate) and queries DynamoDB for all receipts within that range associated with the authenticated user (user_id).
4. **Dynamic Re-rendering:** The API response returns recalculated statistics — including total spending, receipt count, and per-category totals.
5. These values are then propagated to the corresponding React components (e.g., CategoryPieCard, SummaryTile, ChartCard), which re-render automatically to display the filtered analytics.

- **Error Handling**

1. **Missing Parameters:** If any of the required query parameters (`userId`, `fromDate`, or `toDate`) are missing, the system immediately returns a 400 Bad Request response with an error message "Missing parameters".
2. **Invalid Date Range:** When the `fromDate` value is later than the `toDate`, the client rejects the request and throws a validation error before the query is sent, preventing invalid data from reaching the backend.
3. **Query Execution Failure:** If an unexpected error occurs during the DynamoDB query—such as service unavailability, permission issues, or malformed query expressions—the system logs the exception to CloudWatch and returns a 500 Internal Server Error response with a generic message to avoid information leakage.
4. **Empty Result Set:** When no receipts are found within the specified date range, the API still returns 200 OK, but an empty array for `categoryGroups` and a `receiptsCount` of 0. The dashboard then displays an empty-state message instead of showing blank or broken charts.
5. **Network or Request Failure:** If a network error or an API request fails, the dashboard captures the error, halts further updates, and displays an inline red error message in the main content area.
6. **Loading State Management:** While data is being fetched, a temporary loading indicator ("Loading...") is shown to signal progress. Once the response is received—whether successful or empty—the indicator is dismissed, and the dashboard updates accordingly.
7. **CORS Preflight Requests:** The system explicitly handles OPTIONS requests with a 200 OK response and proper CORS headers, ensuring reliable cross-origin communication between the client and API.

- **Data Requirements**

This functionality uses the same backend data source and request structure as 1.3.6 – Visualisation of Expenses, which provides aggregated analytics data from the endpoint

GET /users/{userId}/dashboard?fromDate=YYYY-MM-DD&toDate=YYYY-MM-DD.

The request is authenticated via Cognito and returns per-category spending summaries for the specified date range.

For detailed definitions of required inputs (e.g., `userId`, `fromDate`, `toDate`, and Authorisation headers) and the backend response structure, refer to [Section 5.3.6](#).

In this functionality, the dashboard explicitly uses the same dataset for time-range filtering. When a new start and end date are selected, these parameters are passed to the same `/dashboard` endpoint. The backend re-aggregates category totals and receipt counts for the requested period, and the frontend reuses this updated payload to refresh all visual components — including summary metrics, pie charts, and receipt counts — according to the newly selected time window.

5.3.8. Functionality 8 – Receipt History

This page lists a user’s past receipts, including key fields (date, vendor, total, and item count), and lets the user open a specific receipt for details. Data is pulled from the backend over a fixed history window and normalised so category keys remain consistent across the UI.

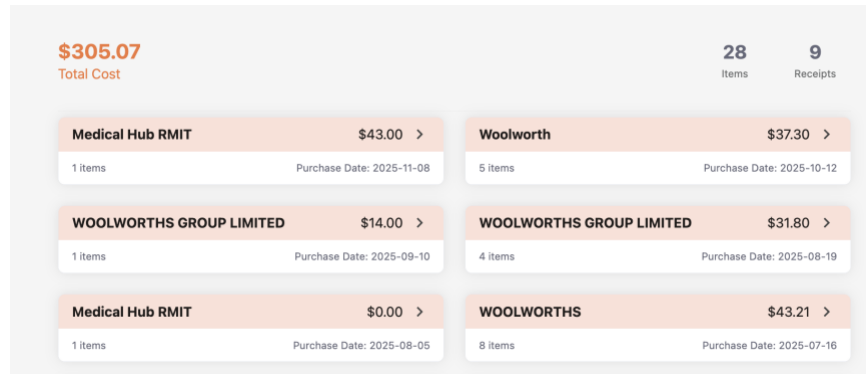


Figure 11 Receipt History page

• Technical Workflow

1. Frontend Request List: When the Receipt History page loads, the client sends a GET `/users/{userId}/receipts?fromDate=...&toDate=...` request with an `Authorisation: Bearer <id_token>` header obtained from Cognito authentication. It applies a default two-year window to retrieve all available historical receipts.
2. Backend → DynamoDB Query: The backend queries the Receipts DynamoDB table using `user_id = {userId}` as the partition key, applying date filters (date BETWEEN fromDate AND toDate) via either the primary key or the `user_date-index`. The query retrieves essential attributes, including receipt_id, date, vendor, total, and items.
3. Backend → Data Normalisation and Aggregation: The backend sanitises numeric values (e.g., converting string totals to numbers), counts each receipt’s `itemsCount = items.length`, and aggregates the total count as `receiptsCount`.
4. Frontend → Process Response and Build State: The client processes the response, converting totals to numeric types and computing summary values such as `totalCost`, `totalItems`, and `totalReceipts`. These are stored in the state variables `summary` and `receipts` for display.
5. Frontend → Render UI: The page renders the top summary section using `summary.totalCost`, `summary.totalItems`, and `summary.totalReceipts`, followed by a list of individual receipts displayed through the `<DashboardGrid>` and `<ReceiptCard>` components. If the list is empty, the page displays a fallback message: “No receipts yet.”
6. Open Details (On Demand): When the user selects a receipt card, the client calls GET `/users/{userId}/receipts/{receiptId}` to fetch detailed receipt data. If the response includes an `items[]` array, the client normalises each item’s category field to match the system’s predefined enumeration before displaying the detailed view.

- **Error Handling**

1. CORS Preflight Requests: If the request is OPTIONS, the endpoint returns 200 OK with CORS headers and an empty body to satisfy browser preflight.
2. Missing Parameters (Backend): If any of `userId`, `fromDate`, or `toDate` is missing, the endpoint returns 400 Bad Request with `{ error: "Missing parameters" }`.
3. DynamoDB Query Failure (Backend): If the `QueryCommand` to DynamoDB throws (e.g., service error, permission issue, bad expression), the handler logs the exception and returns 500 Internal Server Error with a generic payload.
4. Items JSON Parse Fallback (Backend): If `JSON.parse(item.items?.S ?? "[]")` fails for any record, the handler safely falls back to an empty array `[]` for that receipt instead of failing the whole request.
5. Amount Normalisation (Backend): Totals are parsed from numeric (N) or sanitised from string (S) via `sanitiseAmount`, preventing NaN and ensuring a safe numeric value in the response.
6. Environment Misconfiguration (Backend): If `RECEIPTS_TABLE` is unset or incorrect, the DynamoDB call fails and the endpoint returns 500 Internal Server Error.
7. Empty Result Set (Backend): If no receipts match the date window, the endpoint returns 200 OK with `{ receiptsCount: 0, receipts: [] }`, allowing the UI to render an empty state.
8. Missing API Base URL (Frontend): If `NEXT_PUBLIC_API_BASE` (API Gateway base URL) is not configured, the client throws "API base URL is not configured", preventing undefined requests.
9. HTTP Failure (Frontend): Non-200 responses from the history endpoints cause the client to throw with status and response text for diagnosis.
10. Unexpected Response Shape (Frontend): If the list payload is not in the expected shape, the client throws "Unexpected response shape from receipts list endpoint."
11. Empty Body for Details (Frontend): If the detail endpoint returns a falsy body, the client returns null to avoid rendering undefined data.
12. Client-Side Normalisation Guards (Frontend): The client normalises category keys and coerces non-numeric totals/prices to safe defaults to prevent rendering errors in the history view.

- **Data Requirements**

The Receipt History feature relies on aggregated receipt data retrieved from the backend endpoint *GET /users/{userId}/receipts*, scoped to a defined date range. This endpoint does not return detailed item-level data; instead, it provides a compact summary for each receipt, optimised for fast dashboard rendering.

1. Required Inputs:

- `userId` (path parameter, string): The authenticated user's unique identifier (sub) from Cognito.
- `fromDate` (query parameter, string): The start date of the retrieval window in ISO format (YYYY-MM-DD).
- `toDate` (query parameter, string): The end date of the retrieval window in ISO format.
- `Authorisation` (header, string): A valid Bearer token (`id_token`) issued by Cognito, required to authenticate the user.

2. Expected Backend Response Structure:

The backend returns a structured JSON payload in the following format:

```
{ "receiptsCount": 12,
  "receipts": [
    { "receiptId": "abcd1234",
      "date": "2025-09-15",
      "vendor": "Aldi",
      "total": 23.50,
      "itemsCount": 5 } ] }
```

- `receiptsCount` (number): The total number of receipts returned in the query range.
- `receipts` (array): A list of summarised receipt entries, each containing:
 - (1) `receiptId` (string): Unique identifier of the receipt.
 - (2) `date` (string): Purchase date in ISO format.
 - (3) `vendor` (string): The store or merchant name.
 - (4) `total` (number): The overall amount for that receipt, normalised on the backend.
 - (5) `itemsCount` (number): The count of line items within that receipt, computed as `items.length` server-side.

3. Backend Expectations:

- The DynamoDB Receipts table must be indexed by `user_id` (partition key) and, optionally, by `date` (sort key or secondary index).
- The backend must support the date BETWEEN “`fromDate`” AND “`toDate`” filter for efficient query operations.
- Missing or invalid parameters must produce a 400 Bad Request, while successful empty queries must still return 200 OK with empty arrays.
- The response must remain lightweight, excluding `items[]` details — those are only fetched in the separate *GET /users/{userId}/receipts/{receiptId}* endpoint.

5.3.9. Functionality 9 – Single Receipt Detail

This functionality allows users to view the full details of a specific receipt selected from the Receipt History page. It presents a structured overview of the vendor, purchase date, total amount, and the individual line items contained in that receipt. If available, the original scanned receipt image is also displayed alongside the textual data.

From this page, users can navigate back to the history list or enter Edit Mode to modify receipt information.

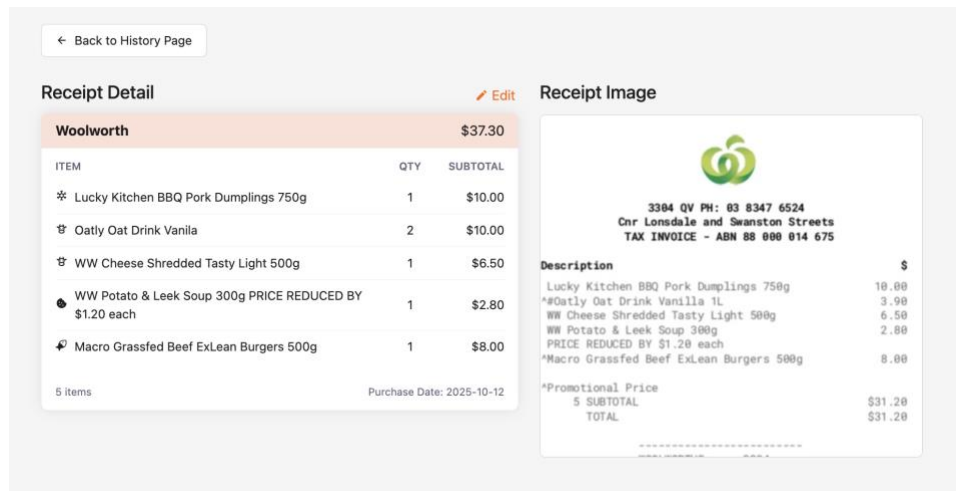


Figure 12 Receipt Detail page

• Technical Workflow

1. **Navigation and Routing:** The page is served under the route `/app/receipt/[receiptId]`. When a user selects a receipt in the History list, the router pushes this dynamic route, passing the `receiptId` as a path parameter. If the user has just saved edits (please refer to [Section 5.3.10](#)), a query parameter `?saved=1` is also appended so that a toast notification can be shown upon redirect.
2. **Data Fetching:** Once both `receiptId` and `userId` are available, the client calls the API helper `fetchReceiptById(userId, receiptId, token)`. This function sends a GET request to `/users/{userId}/receipts/{receiptId}` and returns a normalised Receipt object that meets the data requirements below.
3. **State Management and Lifecycle:** The component maintains local states for receipt, loading, and error. A mounted flag (`alive`) guards against race conditions and prevents state updates after unmount. If the `receiptId` changes or the user navigates to another receipt, the effect automatically re-fetches data.
4. **User Interface Rendering:** The page is wrapped in `<AppLayout>` to maintain consistent navigation and styling. A Back to History button returns the user to `/app/history`. The main content is organised with `DashboardGrid`, dividing the view into two columns:
 - Receipt Details – Rendered by `<ReceiptDetailCard>` showing vendor, date, total, and itemised breakdown.
 - Receipt Image – Displays the uploaded receipt `image.image_url` if available, otherwise a placeholder message.

An Edit button at the top-right triggers navigation to `/app/receipt/[receiptId]/edit`.

5. Toast Notification for Saved Changes: When redirected from the edit page with ?saved=1, the component displays a centred toast reading “Changes saved” for 1.5 seconds, then cleans the query parameter via router.replace(...) without page reload.

- **Error Handling**

1. CORS Preflight: If the request is OPTIONS, the backend returns 200 OK with CORS headers and an empty body to satisfy browser preflight.
2. Missing Path Parameters: If userId or receiptId is missing from the path, the backend returns a 400 Bad Request with { error: "Missing userId or receiptId" }.
3. Receipt Not Found: If the DynamoDB GetItem returns no record, the backend responds 404 Not Found with { error: "Receipt not found" }; the UI renders “Receipt not found.”
4. Items Parse Failure: If items JSON parsing throws, the backend’s try/catch returns 500 Internal Server Error (the UI shows a generic inline error).
5. Image URL Presign Failure: If deriving the S3 bucket/key, or generating a signed URL, fails, the backend returns a 500 Internal Server Error.
6. Unhandled Exceptions: Any other error in the backend handler is logged and returned as 500 Internal Server Error with CORS headers.
7. Missing or Invalid ID: If receiptId is undefined or not a string in the route, the page stops loading and renders “Receipt not found.”
8. API Fetch Failure: If fetchReceiptById throws (e.g., network error or invalid token), the component sets error and renders an inline error message (“Error: Failed to load receipt detail.”).
9. Loading State: While data is being retrieved, a temporary “Loading...” indicator is displayed.
10. Empty Response: If the API returns no data for the specified receiptId, the UI shows “Receipt not found.”
11. Unmount Protection: A cleanup flag prevents state updates after component unmount to avoid React warnings.
12. Toast Display: If the ?saved=1 query parameter is missing or malformed, the toast is suppressed without throwing an exception.

- **Data Requirements**

This functionality depends on complete and correctly structured receipt data retrieved from the backend endpoint GET /users/{userId}/receipts/{receiptId}.

The system requires the following fields for proper rendering and editing:

1. Vendor (string): The store or merchant name.
2. Date (string): Purchase date in ISO format (YYYY-MM-DD).
3. Items (array): Each element must contain name, quantity, price, and category.
4. Total (number): The overall amount, used for validation against recalculated totals.
5. image_url (string, optional): Reference to the receipt image stored in S3.

The data must follow the standard Receipt modal defined in the backend and reflected in the frontend Receipt type.

If any required fields are missing or malformed, the page will either fail validation or display fallback placeholders.

5.3.10. Functionality 10 – Receipt Detail Editing

This functionality enables users to edit the details of an existing receipt, including modifying the vendor name, updating or deleting existing line items, and adding new line items. The edit screen also displays the original receipt image for reference and calculates the gross total in real time as the user updates prices or quantities.

To prevent accidental data loss, the system includes confirmation dialogues for both leaving without saving and saving changes.

Edit Mode Save

Vendor: Woolworth

Items

* Lucky Kitchen BBQ Pork Dumplings 750g	1	\$10.00	>
WW Cheese Shredded Tasty Light 500g	1	\$6.50	>
Macro Grassfed Beef ExLean Burgers 500g	1	\$8.00	>
Oatly Oat Drink Vanilla	2	\$10.00	>
WW Potato & Leek Soup 300g	1	\$2.80	>
PRICE REDUCED BY \$1.20 each			
+ Add New Item			

Gross Total: \$37.30

Receipt Image

3394 QV PH: 03 8347 6524
Cnr Lonsdale and Swanston Streets
TAX INVOICE - ABN 88 808 814 675

Description	\$
Lucky Kitchen BBQ Pork Dumplings 750g	10.00
*Oatly Oat Drink Vanilla 1L	3.90
WW Cheese Shredded Tasty Light 500g	6.50
WW Potato & Leek Soup 300g	2.80
PRICE REDUCED BY \$1.20 each	
*Macro Grassfed Beef ExLean Burgers 500g	8.00

Figure 13 Receipt Detail Edit Mode

- **Technical Workflow**

1. **Accessing Edit Mode:** The user enters edit mode by clicking the Edit button on the Receipt Detail page, which navigates to the route `/app/receipt/{receiptId}/edit`. Upon navigation, the system retrieves the user's Cognito authentication credentials — including the `id_token` and `sub` (unique user ID) — to authorise all subsequent API operations. This ensures that only the authenticated owner of the receipt can modify its data.
2. **Data Retrieval:** Once the edit page loads, it sends a request to the backend endpoint `GET /users/{userId}/receipts/{receiptId}` to retrieve the receipt's existing details. The backend responds with structured data including the vendor name, purchase date, total amount, and an array of individual line items, each containing fields such as item name, quantity, price, and category. An optional `image_url` is provided if the original receipt image is available in storage. After fetching the data, the page stores it in the local state and takes an initial snapshot, which serves as the baseline for detecting unsaved changes during the editing process.
3. **Local Editing Logic:** Within the edit interface, the vendor name is editable through a bound text field that immediately reflects user input in the editor state. The list of items can be modified inline — each row supports editing of name, quantity, unit price, subtotal, and category — or removed entirely if no longer needed. Users can also click “+ Add New Item” to insert a blank editable row, where a new item can be created directly on the page. Each change to an item instantly triggers a recalculation of the gross total, which is updated dynamically using the `calculateTotal()` method to reflect the current in-memory state.

4. **Form Validation:** Before the user can save changes, the system performs real-time form validation to ensure all input fields are complete and valid. The Save button remains disabled until the vendor name is filled in and every line item includes valid name, quantity, price, and category values. When validation conditions are met, the button becomes active, allowing the user to proceed with saving.
5. **Unsaved Change Detection:** The component continuously compares the current editing state with the initial snapshot captured after the first load. Any difference in vendor name, item details, or the presence of an unfinished new item marks the form as “dirty,” activating the unsaved-change guard. If the user attempts to navigate away (for example, by clicking the back button), a modal confirmation dialogue appears, asking whether to leave without saving.
6. **Saving Changes:** When the user clicks the Save button, a confirmation dialogue (“Save changes to this receipt?”) appears to prevent accidental updates. If confirmed, the page invokes the `saveChanges()` method provided by the `useReceiptEditor` hook, which sends a `PUT /users/{userId}/receipts/{receiptId}` request to the backend. The payload includes the updated vendor name and the full array of items, all normalised with numeric values for price and quantity to ensure data consistency. Once the update is successful, the system redirects the user back to `/app/receipt/[receiptId]?saved=1`, where a toast message “Changes saved” confirms successful persistence.

- **Error Handling**

1. **CORS Preflight:** If the request is `OPTIONS`, the backend returns 200 OK with CORS headers and an empty body.
2. **Missing Path Parameters:** If `userId` or `receiptId` is missing, the backend returns a 400 Bad Request with `{ error: "Missing userId or receiptId" }`.
3. **Receipt Not Found:** If no record exists for the specified receipt, the backend returns a 404 Not Found status code; the UI displays “Receipt not found.”
4. **JSON Parsing Failure:** If the JSON deserialisation of the items fails during retrieval or update, the backend’s `try/catch` returns a 500 Internal Server Error.
5. **Update Validation Error:** If the request body omits required fields (e.g., vendor is missing or items are malformed), the backend rejects the update with a 400 Bad Request.
6. **DynamoDB Update Failure:** If the DynamoDB write operation fails (e.g., due to permission or service errors), the backend logs the error and returns 500 Internal Server Error.
7. **Missing or Invalid ID (Frontend):** If `receiptId` is undefined or invalid, the page stops loading and shows “Receipt not found.”
8. **Fetch or Save Failure (Frontend):** If API requests fail, the UI displays an inline error message (“Save failed.”) and remains in edit mode.
9. **Validation Block:** The Save button remains disabled until all mandatory fields are filled; users are alerted when attempting to save incomplete entries.
10. **Unsaved Change Confirmation:** If the user attempts to leave with pending edits, a modal dialogue appears asking for confirmation.
11. **Unmount Safety:** The page avoids updating state after unmount to prevent React warnings during navigation transitions.
12. **Missing Image:** If no image is available, a fallback message, “No image available for this receipt,” is displayed.

- **Data Requirements**

This functionality relies on a structured receipt payload provided by the backend endpoint GET /users/{userId}/receipts/{receiptId} and updates data via PUT /users/{userId}/receipts/{receiptId}.

1. Required Inputs and Identifiers:

- receiptId (path param, string): The unique identifier of the target receipt.
- userId (path param / auth context, string): The currently authenticated user (Cognito sub).
- token (Authorisation Bearer, string): The Cognito OIDC id_token, used for backend authorisation.

2. Data Structure – Receipt Detail (GET response):

- vendor (string): The merchant or store name.
- items (ReceiptItem[], optional but usually present): The main editable dataset. Each ReceiptItem must contain:
 - (1) name (string): The item name.
 - (2) quantity (number): The purchased quantity; must be greater than 0.
 - (3) price (number): The line subtotal (not a unit price); must be greater than 0.
 - (4) category (string): The expense category key (matches system enumeration).

3. Data Structure – Save Request (PUT request body):

- vendor (string, required): Must be non-empty; otherwise blocked by frontend validation.
- items (ReceiptItem[], required): Every item must pass validation (non-empty name and category, numeric quantity > 0 and price > 0).
- total (number, required): Computed on the client using calculateTotal(items), which sums all item.price values (each already representing the full line subtotal).

4. Frontend Validation Rules (enforced by the editor):

- Before saving, isValid() ensures that the vendor is filled and all items meet the numeric and non-empty constraints; otherwise, the save request is blocked.
- Prior to submission, each item.price is coerced into a safe numeric value (NaN converted to 0), and the total is recalculated using calculateTotal().
- If the GET response is empty or invalid, the editor resets its state (receipt = null, items = [], vendor = "") and shows a fallback error or empty state.

5. Backend Expectations:

- The backend must return a structured receipt object with the fields above. Missing mandatory fields should be treated as an invalid state by the client.
- The PUT endpoint must accept vendor, total, and items, and perform server-side validation of numeric fields and required attributes.
- On success, the API should return 200 OK (or an equivalent success code).
- In case of parsing, validation, or permission errors, appropriate HTTP error codes (400, 404, 500) should be returned. The frontend will remain in edit mode and display the corresponding error message.

5.4. Database and Storage Architecture

The ReceiptCAT system utilises AWS-managed storage services to ensure scalability and reliability. Structured receipt and metadata records are stored in Amazon DynamoDB, while unstructured assets such as uploaded receipt images and temporary processing files are stored in Amazon S3. Together, these components form a secure and serverless data foundation for all application workflows.

5.4.1. Database – Amazon DynamoDB

The ReceiptCAT backend uses a single Amazon DynamoDB table to store all processed receipt data. This table is defined and provisioned within the receiptcat-infra repository (in the file: backend-infra-stack.ts) using AWS CDK, ensuring consistent deployment across environments (development, feature, and production).

The table, named according to the pattern: <app>-<domain>-<stage>-receipts (e.g., receiptcat-backend-dev-receipts), serves as the central data source shared between the Textract-based Receipt Extractor Lambda and the Bedrock-based Receipt Categoriser Lambda. Both Lambdas access the same table via the environment variable RECEIPTS_TABLE, which is automatically injected at deployment time

5.4.1.1. Table Configuration

The DynamoDB table that underpins the ReceiptCAT backend is designed to support multi-user isolation, efficient query performance, and scalable data growth. Each record represents a single processed receipt associated with a verified Cognito user.

The table schema uses a composite primary key (user_id, receipt_id) to uniquely identify receipts per user. At the same time, a Global Secondary Index (GSI) enables quick retrieval of user-specific data, such as querying by transaction date.

The configuration parameters are summarised below:

Property	Value / Description
Partition Key	user_id (<i>String</i>) — the unique Cognito user identifier that isolates receipt data per user.
Sort Key	receipt_id (<i>String</i>) — the unique ID generated for each uploaded receipt.
Global Secondary Index (GSI)	user_date-index — allows efficient querying of receipts by user and receipt date.
Billing Mode	Provisioned (auto-scaling read/write capacity between 1–10 units with 70% utilization target).
Removal Policy	DESTROY for development stages; RETAIN for production deployments.

Table 7 DynamoDB Table Configuration

5.4.1.2. Table Schema

Each record in the Receipts Table represents a single processed receipt and contains the following attributes:

Attribute	Type	Description
user_id	String	Cognito user identifier (Partition Key).
receipt_id	String	Unique ID for each processed receipt (Sort Key).
vendor	String	Merchant name extracted from the receipt (e.g., <i>Aldi</i> , <i>Coles</i>).
date	String	Transaction date in ISO format (YYYY-MM-DD).
total	Number	Total amount of the transaction.
items	String (JSON)	JSON array containing individual line items with name, price, quantity, and later category.
s3_path	String	S3 URI to the stored receipt image file.
processed_at	String	Timestamp of when the receipt was processed by the Extractor Lambda.

Table 8 DynamoDB Table Configuration

This schema enables a per-user, multi-receipt model, allowing fast retrieval of receipts by user or date while ensuring strong data isolation between users.

By sharing the same table between the extraction and categorisation stages, the architecture reduces redundancy, simplifies integration, and supports efficient dashboard and history queries through the user_date-index.

5.4.1.3. Data Flow Integration

This section explains how the DynamoDB table integrates with the AI processing pipeline to maintain a unified and consistent record for each uploaded receipt. Both the extraction and categorisation stages interact with the same table to ensure that raw data, intermediate results, and AI-generated outputs are stored under a single consolidated item.

The integration flow is summarised below:

- **Receipt Extractor Lambda (Textract stage)**

When a new receipt image is uploaded to S3, the extractor Lambda parses the image using Amazon Textract and writes a new record into this table via PutItem. It populates base fields such as vendor, total, items, and s3_path.

- **Receipt Categoriser Lambda (Bedrock stage)**

The categoriser Lambda later retrieves the same record using (user_id, receipt_id) as the composite key and updates it with AI-predicted categories via UpdateItem.

This ensures that both raw extraction data and categorised results remain consolidated under a single DynamoDB item.

5.4.2. S3 Bucket Structure Design

The system leverages Amazon S3 as the primary object storage service for managing user-uploaded data. To ensure multi-tenant isolation, logical separation of data types, and scalability, all objects are stored under a well-structured prefix hierarchy. Each user is uniquely identified by a subclaim from AWS Cognito (a UUID-like identifier).

5.4.2.1. Directory Layout:

The S3 bucket receiptcat-backend-dev-receipts is organised into a clear, user-isolated directory structure designed to support both temporary and long-term storage of receipt images. Each user's data is securely contained under their unique Cognito sub-identifier, ensuring strict access isolation and traceability.

The overall directory layout is as follows:

```
s3://receiptcat-backend-dev-receipts/
├── tmp/users/{sub}/receipts/{timestamp}_{uuid}_{safeName}.jpg  # Temporary upload area
├── users/
│   └── {sub}/
│       └── receipts/
│           └── {timestamp}_{uuid}_{safeName}.jpg
```

Figure 14 S3 Structure Design

5.4.2.2. Folder Definitions

The folder organisation follows a clear separation of temporary, user-specific, and permanent storage areas to support efficient processing, secure data isolation, and compliance with lifecycle management policies.

- **tmp/ folder:**

- Purpose: Serves as the temporary upload area for receipt images before processing.
- Each upload is placed under the corresponding user's subdirectory, where {sub} represents the Cognito User Pool unique identifier. Files stored here are short-lived and automatically deleted by S3 lifecycle policies after 24 hours. Objects in this directory trigger the Textract processing Lambda, which extracts structured data from the uploaded receipt and subsequently moves the file to its permanent location under users/{sub}/receipts/.

- **users/{sub}/ folder:**

- Each user is assigned a dedicated namespace under the users/ directory, ensuring strict isolation of user data and preventing overlap between tenants.
- This directory contains multiple subfolders, each serving a distinct functional purpose.

- **receipts/ folder:**

- Purpose: Permanent storage for validated receipt images.
- Files are moved here after backend validation/processing.
- Provides a clean separation between raw uploads (in tmp/) and confirmed records.

5.4.2.3. Security and Compliance Considerations

This section outlines the security configurations and compliance controls applied to the receiptcat-backend-dev-receipts S3 bucket.

These measures ensure that all stored and transmitted data adhere to AWS security best practices, maintaining confidentiality, integrity, and availability throughout the entire data lifecycle. The configuration enforces strict access control, encryption, and automated lifecycle management to meet both operational and regulatory requirements.

- **Authentication and Authorisation**

All access to S3 objects is strictly controlled through Amazon Cognito authentication. Users must be authenticated via Cognito before obtaining a presigned URL, which authorises a short-lived, single-purpose PUT request to a specific object key.

Each object key is prefixed with the user's Cognito sub identifier (e.g., tmp/users/{sub}/... or users/{sub}/...), ensuring tenant-level isolation and preventing cross-user access.

Public access is fully blocked, and all requests must be made over HTTPS.

- **Encryption**

Server-side encryption using Amazon S3–managed keys (SSE-S3) is enabled by default, ensuring that all objects are encrypted at rest. In addition, the bucket enforces SSL-only access (enforceSSL: true) to guarantee data is encrypted in transit.

The configuration complies with AWS security best practices and protects user data during both upload and retrieval operations.

- **Lifecycle and Data Retention**

The bucket implements automated lifecycle management policies to balance security, cost, and data governance:

- Temporary uploads tagged with stage=tmp (under tmp/users/{sub}/...) are automatically deleted after 24 hours, ensuring transient data does not persist beyond processing needs.
- Incomplete multipart uploads are automatically aborted after 1 day to prevent orphaned uploads from consuming storage.
- Non-current object versions expire after 30 days to maintain versioned storage hygiene.
- Processed data under users/{sub}/receipts/, profile/, and exports/ is retained for long-term storage, unless explicitly deleted by the user or removed through higher-level data retention policies.

- **Access Control and Least Privilege**

AWS IAM roles and bucket policies enforce the principle of least privilege, granting Lambda functions and backend services only the permissions necessary for their respective operations (e.g., read/write within specific prefixes).

No direct public access is allowed, and all cross-service communication occurs within the secured AWS environment.

5.5. Non-functional specifications

The non-functional characteristics of the ReceiptCAT system are primarily ensured by its underlying AWS serverless architecture, which inherently delivers high reliability, scalability, and security.

The system has been designed in alignment with standard software engineering best practices, leveraging AWS-managed services to ensure operational robustness, maintainability, and consistent performance across all deployment environments.

5.5.1. Privacy and Data Protection

Privacy protection is a key consideration in the design of the ReceiptCAT system, as uploaded receipts may contain personally identifiable information (PII) such as names, addresses, card numbers, or contact details.

To mitigate this risk, both the AI processing pipeline and the user interface have been designed to minimise unnecessary data collection and prevent exposure of sensitive information.

At the AI processing level, the system strictly limits data extraction to essential fields only—vendor name, item name, prices, totals, and purchase dates. No personal identifiers, such as phone numbers, payment card details, or addresses, are stored or transmitted. The AI models (Amazon Textract and Bedrock) operate on transient data within the secured AWS environment, and outputs are sanitised before being persisted in DynamoDB.

At the user interface level, the preview page provides [Section 5.3.2](#), allowing users to manually redact or remove any parts of a receipt they do not wish to share before uploading. These edits are applied locally in the browser, ensuring that unredacted data never leaves the user's device.

Together, these measures ensure that ReceiptCAT complies with privacy-by-design principles, protecting user confidentiality throughout the entire receipt processing workflow.

5.5.2. Security

Security is a core consideration in the design of the ReceiptCAT system.

User authentication and authorisation are fully managed by Amazon Cognito, ensuring that all access to the application and backend APIs is securely verified. The authentication flow supports both federated sign-in with trusted third-party providers (e.g., Google) and traditional email–password authentication. In the latter case, Cognito enforces email verification before granting access, ensuring that only verified accounts can interact with the system.

Each authenticated user is assigned a unique UUID and session token, which are propagated across subsequent API calls and secured pages under the /app route. All API Gateway endpoints are protected using Cognito authorizers that validate the user's token before processing any request, preventing unauthorised access to personal receipt data. This architecture ensures strong identity isolation, robust access control, and end-to-end security from login to data retrieval.

5.5.3. AI Categorisation Accuracy

The performance of the AI-based categorisation component has been thoroughly evaluated to ensure its reliability and consistency within the ReceiptCAT system.

Based on internal testing, the categorisation model achieved an accuracy rate of up to 95.4%, depending on the input quality and prompt configuration used during processing. This high level of precision demonstrates the robustness of the adopted prompting strategy and the suitability of the selected model for real-world receipt data.

Further experimental results, including comparative analyses of few-shot vs. no-shot configurations, model response consistency, and accuracy metrics across different AI models, are documented in the supplementary research report P000340SE–EvaluationReport–Bedrock. This report provides a detailed discussion of the model’s performance under various operational conditions and its integration within the production pipeline.

5.5.4. Performance

The overall performance of the ReceiptCAT system was assessed based on key user-facing and backend operational metrics, focusing on upload responsiveness, AI processing time, and dashboard rendering latency. The system’s serverless architecture and asynchronous, event-driven workflow ensure that most AI-related processing occurs independently of the user session, maintaining a responsive user interface.

5.5.4.1. Textract Processing Performance

Performance data for the text extraction stage was obtained directly from AWS CloudWatch Metrics for the Lambda function that invokes Amazon Textract (AnalyzeExpense).

The observed dataset covers a two-month period (September and October 2025) and provides a representative measurement of typical system behaviour under production-like usage.

Metric	Value	Description
Minimum Duration	2.01 s	Fastest observed execution, representing optimal conditions with minimal cold-start delay and small receipt size.
Average Duration	2.21 s	Typical end-to-end Lambda execution time, including Textract API call, result parsing, and DynamoDB persistence.
Maximum Duration	2.36 s	Longest observed execution, demonstrating low variance and consistent performance across all invocations.

Table 9 Textract service two-month performance metrics

These results indicate that the Textract extraction pipeline consistently completes within approximately 2–2.5 seconds per receipt, including all Lambda orchestration overhead. Such consistency reflects the efficiency of AWS’s managed AI service integration and the lightweight structure of the receipts being processed.

Data was sourced from the CloudWatch “Duration” metric for the TextractProcessorLambda function, using the p50 (average) aggregation over a two-month observation window. This confirms that the text extraction stage operates well within acceptable latency thresholds for near-real-time user feedback in the dashboard.

5.5.4.2. Bedrock Categorisation Performance

Performance metrics for the AI categorisation Lambda function (responsible for invoking Amazon Bedrock – Claude 3.7 Sonnet) were collected from AWS CloudWatch over a two-month observation window (September–October 2025).

The measurements reflect the end-to-end Lambda execution time, including Bedrock model invocation, JSON parsing, and DynamoDB update operations.

Metric	Value	Description
Minimum Duration	1.73 s	Fastest observed end-to-end execution.
Average Duration	1.73 s	Typical execution time, showing negligible deviation across all invocations.
Maximum Duration	1.73 s	Longest observed duration, confirming consistent Bedrock performance across multiple runs.

Table 10 Bedrock service two-month performance metrics

These results demonstrate that the categorisation stage is highly consistent and lightweight, averaging approximately 1.7 seconds per receipt from Lambda trigger to DynamoDB update completion. Such stability indicates minimal cold-start effects and efficient model inference via the managed Bedrock API.

The performance data was sourced from CloudWatch Duration metrics for the BedrockCategoriserLambda function, aggregated over a two-month period using the Average (p50) statistic. This confirms that the Bedrock model delivers near real-time categorisation capabilities within the overall AI processing workflow.

5.5.4.3. Upload and Dashboard Latency

Client-side performance was evaluated under different network environments to measure user-perceived responsiveness during critical interactions—receipt upload and dashboard data rendering.

All measurements were conducted using Chrome DevTools Network Throttling and verified under stable conditions to reflect realistic user experiences.

Component	Condition	Average Duration	Observations
Dashboard Data Loading	No Throttling	0.99–1.07 s	Standard load time from page navigation to full visualisation rendering.
	Fast 4G	1.09–1.20 s	Slight delay due to network latency; UI remains smooth and responsive.
	Slow 4G	1.82–1.92 s	Noticeable but acceptable delay; loading indicator ensures user feedback.
Receipt Upload (233 KB image)	No Throttling	≈ 3.36 s	Average total time from “Confirm Upload” click to redirect after completion.
	Fast 4G	≈ 5.20 s	Slightly longer due to presigned URL retrieval and upload transfer time.
	Slow 4G	≈ 7.63 s	Maximum observed delay; system remains stable with progressive status updates.

Table 11 Dashboard rendering and Upload service performance metrics

Upload duration increases linearly with file size, with the upper bound tested at 5 MB, showing proportionate latency growth but no failed uploads.

During uploads, all other interface interactions are disabled to maintain data integrity, and a success toast is displayed immediately after completion.

Similarly, dashboard loading maintains sub-2-second responsiveness under most network conditions, supported by efficient API payload design and incremental component rendering.

5.5.5. Scalability and Reliability

The scalability and reliability of the ReceiptCAT system are primarily ensured through its fully serverless deployment on AWS, leveraging managed services that automatically handle traffic surges and fault recovery without manual intervention. Each major component contributes to operational stability and elastic scaling:

- Amazon CloudFront acts as the global content delivery network (CDN) for the front-end application, caching static assets across AWS edge locations worldwide. This ensures low-latency page loading and high availability for users regardless of geographic region.
- Amazon Cognito provides identity management with auto-scaling user pools, enabling secure and reliable authentication even under high concurrent sign-in loads. Token issuance and verification are distributed across multiple Availability Zones for fault tolerance.
- AWS Lambda offers on-demand compute scalability, automatically handling fluctuating workloads for both the Textract and Bedrock processing functions while maintaining consistent response times.
- Amazon API Gateway scales horizontally to process incoming API traffic, with built-in throttling, caching, and integration with Cognito authorisers to ensure secure, reliable routing.
- Amazon DynamoDB ensures consistent, low-latency data access through automatic partition scaling and multi-AZ replication, maintaining durability and high throughput for read and write operations.
- Amazon S3 provides highly durable object storage (99.999999999% data durability) with multi-region redundancy, guaranteeing long-term accessibility of receipt images and processed outputs.
- Amazon CloudWatch continuously monitors service health, latency, and error metrics, enabling proactive performance tuning and early anomaly detection.

Through this combination of AWS-managed scalability and a fault-tolerant design, ReceiptCAT achieves near-instant scaling, high data durability, and zero downtime across its end-to-end processing pipeline.

This architecture ensures the system can seamlessly accommodate growth in user volume while maintaining stable, predictable performance in production.

5.6. Known Issues & Limitations

While the ReceiptCAT system performs reliably under most operating conditions, several known issues and inherent limitations have been identified through testing and real-world usage, primarily related to text extraction accuracy and data interpretation within AI services.

5.6.1. Textract Recognition Limitations

This section outlines the current limitations and known recognition issues of Amazon Textract (AnalyzeExpense) within the ReceiptCAT pipeline.

While Textract provides reliable optical character recognition (OCR) and structured data extraction for standard receipt formats, certain edge cases and template inconsistencies can result in partial or inaccurate data capture.

These limitations primarily arise from variations in receipt layouts, inconsistent date conventions, and non-textual vendor information.

- **Date Misinterpretation:**

Amazon Textract occasionally misinterprets date formats due to regional and layout variations across different receipt templates. For example, a date printed as 11/10/2025 may be incorrectly parsed as 10 November 2025 instead of 11 October 2025. This can cause certain receipts to appear as future transactions in the dashboard and receipt history view.

- **Vendor Name Detection:**

Some retailers (e.g., *Woolworths*, Chemist Warehouse) display their brand logos on receipts instead of textual vendor names. In such cases, Textract may fail to detect the vendor field or incorrectly substitute it with unrelated text elements from the receipt header.

- **Non-Standard Receipt Layouts:**

Receipts with unconventional structures—such as Wi-Fi bills, medical invoices, or utility payment statements—may not conform to the standard item–price–total format expected by the extraction pipeline. As a result, certain key fields (e.g., total amount or item descriptions) may be missing or inaccurately extracted, requiring manual correction through the edit interface.

- **Discount Line Misclassification:**

When discounts or promotions are printed as separate lines rather than integrated into item subtotals, Textract may interpret them as independent items. This can lead to duplicated or inflated line-item counts in the extracted data, though users can manually amend these entries in the editing mode.

5.6.2. System Limitations

In addition to AI extraction constraints, several functional limitations exist across the system's user-facing components. These limitations reflect the project's scope boundaries and the technical trade-offs made to ensure it is completed within the allocated timeframe.

5.6.2.1. Preview Page Constraints

This section outlines the current limitations of the Preview Page, which handles user uploads and image editing before AI processing.

While the interface provides essential features such as cropping, blackout, and reset functionality, it presently supports only basic image-based workflows due to time and scope constraints.

- **File Type Support:**

The preview and editing interface currently supports only image-based uploads in JPG, JPEG, and PNG formats.

- Limitation: PDF receipts are not supported.
- Reason: Handling PDF inputs would require integrating separate AWS processing tools (such as Amazon Textract Document API or third-party PDF parsers) to convert multi-page and vector-based layouts into image or text representations before editing. The additional backend logic and increased testing complexity could not be implemented within the available project duration.

- **Multi-File Upload:**

The system supports uploading and processing one receipt at a time.

- Limitation: Multi-file batch upload is not available.
- Reason: Implementing concurrent uploads would require redesigning the upload controller, S3 presigning logic, and state management for multiple file objects. This feature was deprioritised to maintain stability and ensure accurate single-receipt processing for AI analysis.

5.6.2.2. Receipt Management Constraints

This section describes limitations related to the Receipt History and Receipt Management functionalities.

The current implementation focuses on reliable viewing and editing of existing receipts, but does not include advanced data management features such as deletion or custom categorisation.

- **Uploaded receipts cannot be deleted within the receipt history page:**

Once the receipts are uploaded to the system, it cannot be manually removed from client side

- Limitation: The system does not currently support post-upload deletion or date metadata editing.
- Reason: Adding deletion and update functionality would require extending the backend API to handle data consistency and historical audit logging, introducing

additional access control complexity. Given the limited time for backend API expansion and validation, this was excluded from the project scope.

- **Custom Category Management:**

Receipt categorisation is restricted to the predefined set generated by the AI model.

- Limitation: Users cannot add or rename custom expense categories.
- Reason: Implementing dynamic user-defined categories would require schema adjustments in DynamoDB and additional logic to merge or reclassify existing AI-generated records. The design team prioritised stable AI-based categorisation accuracy over advanced user customisation for this delivery phase.

5.6.2.3. Dashboard Constraints

This section summarises the constraints of the Dashboard Page, which presents the aggregated expense analytics and category-based visualisation.

Although the dashboard provides a fully functional interactive pie chart with date filtering, several advanced analytical features remain unimplemented.

- **Visualisation Options:**

The dashboard visualises expenses exclusively through a single pie chart.

- Limitation: Additional chart types (e.g., bar charts, line graphs) and sorting options (e.g., by vendor or amount) are not yet available.
- Reason: Implementing multiple chart modes would require extending the data processing logic in the frontend and designing additional UI components. Given the focus on completing the main interactive pie chart with full date filtering integration, expanded analytics views were deferred.

- **Data Export Features:**

The dashboard does not currently support exporting data or reports.

- Limitation: No built-in options exist for exporting visualised analytics as PDF or CSV.
- Reason: Implementing data export would involve client-side rendering of chart components into downloadable documents (using libraries such as html2canvas or jsPDF) and additional UI controls for report generation. Due to time and testing constraints, this enhancement was not implemented in

Test Planning and Result

6. TEST PLANNING AND RESULT

6.1. Introduction

Testing is conducted continuously across all sprints to ensure reliability, usability, and correct functionality. This document outlines the testing strategy, scope, and procedures for both frontend and backend components.

6.1.1. Objectives

The major objectives of this test plan are as follows: .

- Communicate the testing strategy and process to stakeholders.
- Detect and resolve defects early via automated and manual testing.
- Verify key workflows (receipt upload → extraction → categorisation → dashboard display).
- Ensure AWS integrations (Textract, Bedrock, S3, DynamoDB) perform correctly.
- Validate frontend usability, responsiveness, and cross-browser compatibility.
- Maintain continuous quality throughout all five development sprints.

6.1.2. Tasks

- **Frontend:** Component and unit testing using Jest and Cypress for end-to-end (E2E) workflows.
- **Backend:** Unit and integration testing using Jest for all Lambda functions.
- Manual usability testing for both frontend and backend workflows.
- Cross-browser and responsive testing for frontend UI.
- Code reviews and static analysis to maintain code quality.

6.2. Test Items

The following table outlines the primary components and modules tested as part of the ReceiptCAT system. Unit tests were implemented across both backend and frontend components to verify individual functionalities and logic. Functional tests were conducted manually to validate key user flows and system behaviour in real usage scenarios. Each item below lists the tested component, its version, and relevant notes on its purpose or functionality within the overall system.

Component / Lambda	Version	Notes
Dashboard Lambda	1.0	Backend core feature
Receipt Details Lambda	1.1	Backend core feature
Receipt History Lambda	1.2	Backend core feature
Update Receipts Lambda	1.3	Backend core feature
Presign Upload Lambda	1.4	Backend support
Receipt Categoriser Lambda	1.5	Backend AI categorisation
Receipt Extractor Lambda	1.6	Backend OCR extraction
Receipt Utils / Handler	1.7	Minor backend helper tests
Frontend React Components	1.8	To be detailed later (unit tests & UI components)
Cypress E2E Test Suite	1.9	Covers key user workflows

Table 12 Test Items

6.3. Features to be tested

The following features represent the key **backend and frontend components** that underwent **unit testing** to verify their functionality, reliability, and error handling. Each feature was tested in isolation to ensure correct data flow, user interaction, and integration between system layers.

Feature	Expected Functionality	Risk/Priority
Receipt Extractor Lambda	- Processes S3 events for new receipts. - Validates S3 object key format; rejects invalid keys. - Retrieves S3 objects and handles missing files. - Sends documents to Textract; handles failures. - Extracts total, date, vendor, line items; normalises totals and dates. - Persists receipt data to DynamoDB. - Publishes notifications via SNS.	High - Core ingestion, failure blocks all receipts.
Receipt Details Lambda	- Returns full receipt details for a given receiptId. - Parses totals, date, vendor, items. - Handles missing or malformed data gracefully. - Returns 404 if receipt not found; 500 on server error.	High - Users rely on accurate detail view.
Update Receipt Lambda	- Updates receipt fields (vendor, date, total, items) via PUT. - Validates userId and receiptId presence. - Rejects invalid or missing update fields. - Updates DynamoDB; handles errors gracefully. - Handles CORS OPTIONS requests. - Returns HTTP codes 200, 400, 404, 500 as appropriate.	High - Must maintain data integrity, failure can corrupt receipts.
Receipt Categorizer / buildCategoryGroups	- Groups items from receipts by category. - Aggregates totals per category. - Handles missing quantities (defaults to 1). - Returns empty list for no receipts. - Handles malformed items gracefully.	High - Critical for analytics/dashboard, non-blocking for ingestion.

AppHeader Component	• Renders header for desktop and mobile. • Handles navigation and menu selection. • Triggers onMenuOpen (mobile) and onLogout (desktop). • Highlights active page and supports right-slot content.	High - Core navigation, failure disrupts user access.
Editable Item Row Component	• Shows an item's name, quantity, price, and category in a row. • Switches between display mode and editable mode when clicked or via keyboard. • Allows editing of name, quantity, price, total, and category. • Automatically recalculates unit price when total or quantity changes. • Validates inputs and disables "done" for invalid entries. • Supports adding new items, cancelling edits, and deleting items with confirmation. • Handles missing data or invalid numbers gracefully. • Updates parent via callbacks (onChange, onDone, onDelete).	High - Core part of receipt editing, errors affect data integrity and UX.
Preview Editing Button Component	• Handles image editing actions: blackout, crop, and reset. • Syncs internal state with props (blackoutActive, cropActive). • Enforces mutual exclusion between blackout and crop actions. • Respects disabled state to prevent interaction. • Renders appropriately for desktop (always visible) and mobile (inside a toggleable menu). • Initializes tooltip infrastructure for hover guidance. • Updates ARIA states for accessibility. • Visually reflects action state via icons and styling.	High - Core editing functionality, failure prevents users from modifying previews or interacting correctly with the editing UI.
Preview Editing Tool Component	• Renders the image editing canvas with integrated Cropper (mocked for testing). • Exposes an imperative API for crop and blackout actions that delegates to internal core handlers. • Handles crop mode state and aspect ratio mapping (e.g., "free" → NaN). • Supports restoreState for crop data and crop box, with error-resilient callbacks.	High - Core UI for editing, failure prevents users from cropping or blacking out images properly, affecting all downstream preview workflows.

	• Displays and manages blackout overlays (committed and draft rectangles). • Handles pointer events for blackout interactions (down, move, up, leave, cancel). • Shows cursor hints appropriately for crop mode. • Merges base styles and forwards root attributes correctly.	
Preview Image Frame Component	• Displays a preview of the uploaded image with file name and size. • Allows users to reselect a different file via hidden input. • Integrates image editing via ImageEditingCanvas. • Applies edits and propagates updated image to parent handler. • Handles errors gracefully when edits fail.	High - Core UI for uploading and editing receipts, failure prevents user from providing usable images.
Receipt Card component	• Displays receipt vendor, total amount, number of items, and purchase date. • Handles missing or empty item arrays gracefully. • Navigates to the detailed receipt page on click. • Formats totals and item counts for readability. • Serves as a clickable summary card for the receipts list.	High - Core UI element for browsing receipts, failure blocks basic receipt viewing.
Receipt Detail Card component	• Displays vendor name, total cost, and purchase date. • Lists all receipt items with name, quantity, subtotal, and category icon. • Defaults quantity to 1 if missing and handles unknown categories gracefully. • Shows total item count in the footer. • Supports empty states when no items are provided	High - Core detail view for receipts, incorrect rendering affects user understanding of purchased items.
Require Auth component	• Wraps protected content and enforces authentication. • Shows loading state while authentication is in progress. • Displays error messages if authentication fails. • Redirects or shows "Redirecting..." when user is not authenticated or token is expired. • Renders children normally when user is authenticated.	High - Critical for app security; incorrect behavior could expose protected content or block legitimate users.

App Layout	• Renders responsive header and mobile drawer, highlighting current menu based on route. • Supports navigation when menu items are selected. • Provides an Upload button to pick files, store in session, and navigate to preview. • Handles logout securely and gracefully, including errors. • Can disable header interactions when needed.	High - Core app shell, failures block navigation, logout, and upload functionality.
Edit Receipt Page	• Fetches receipt by receiptId and displays vendor, items, total, and image. • Supports inline editing of existing items. • Allows adding new items with validation. • Supports deleting items. • Dynamically calculates gross total. • Handles Save workflow: confirmation modal, API persistence, error handling, navigation back with saved flag. • Warns before leaving with unsaved changes. • Preserves snapshot state across re-renders. • Keyboard accessible triggers for adding items and other interactions. • UI polish: hover effects, confirmation dialogs.	High - Core to receipt management, directly affects data integrity and UX, failure prevents users from editing receipts reliably.
Receipt Detail Page	• Fetches and displays receipt details based on receiptId. • Shows loading, error, and “not found” messages appropriately. • Displays receipt image or fallback if missing. • Provides buttons to navigate back or edit the receipt. • Shows a toast if returning from edit with a saved flag	High - Core page for viewing receipts, failures block users from seeing or editing receipt data.

Dashboard Page	• Displays greeting with user name, email, or fallback. • Shows topline statistics: total spend, number of items, number of receipts. • Lists categories with CategoryCard components and visualizes spend with a CategoryPieCard. • Handles empty state, loading, and error messages gracefully. • Wraps content in AppLayout and provides responsive layout.	High - Core landing page, failure blocks users from seeing summaries, categories, and overall app overview.
History Page	• Shows summary of total spend, total items, and number of receipts. • Lists all user receipts as ReceiptCards. • Handles empty state with a “No receipts yet” message. • Displays loading indicator when fetching data. • Shows error message if data retrieval fails.	High - Core for users to review past receipts, failure blocks access to historical data.
Login Page	• Redirects authenticated users to /app or the URL from NEXT_PUBLIC_OIDC_REDIRECT_URI. • Calls signinRedirect for unauthenticated users when no active navigator exists. • Does not trigger redirect if an auth navigator is already active. • Handles errors from signinRedirect gracefully (logs them). • Renders without crashing.	High - Controls user access and entry point, failure prevents users from logging in.
Preview Page	• Shows a preview of the uploaded file (image) with its name and size. • Allows user to crop, blackout, or reset edits via editing controls. • Disables the “Confirm” button while edits are active or uploading. • Calls controller functions for confirm, back, reselect, store edits, and reset actions. • Displays error messages for invalid file types or upload errors, and shows a success toast after confirm.	High - critical for correct preview/edit/upload workflow.

Receipt History Lambda	- Retrieves all receipts for a user. - Supports fromDate / toDate filters. - Defaults missing totals to 0; handles invalid items JSON. - Handles CORS OPTIONS requests. - Returns proper HTTP codes (200, 400, 500).	Medium - Users rely on historical data, UX critical.
Dashboard Lambda	- Retrieves user receipt summaries. - Aggregates totals, item counts, and optionally category breakdown. - Defaults missing or malformed data to safe values. - Handles DynamoDB failures gracefully; returns HTTP 500 if needed.	Medium - Enhances user experience, core functionality intact without it.
Pre-sign Upload Lambda	- Generates pre-signed S3 URLs for uploads. - Validates request parameters. - Handles S3 errors gracefully. - Returns URL for client upload.	Medium - Security sensitive, failure prevents uploads but not existing data.
Category Card Component	- Displays category title, icon, and formatted total. - Renders list of items with name and price. - Handles empty category gracefully. - Integrates with CollapsibleCard for expand/collapse	Medium - Impacts dashboard clarity, non-critical to core flow.
Chart Card (Category Pie Card) Component	- Displays spend by category as pie chart. - Shows total and legend with % values. - Allows toggling categories and resetting legend. - Handles compact layout and empty data. - Clamps chart size for responsiveness.	Medium - Impacts analytics UI, not core functionality.
Chart Card Legend Component	- Renders a pill per category with icon & color. - Toggles visibility via click/keyboard. - Shows Reset when all hidden. - Normalizes source labels.	Medium - affects chart visibility.
Collapsible Card Component	- Renders header, body, optional icon/extra, and chevron (if collapsible). - Toggles expand/collapse via click/keyboard; defaultExpanded sets initial state.	Medium - affects UX and accessibility.

	• Applies accessible attributes (aria-expanded, aria-controls, role="region"). • Calls onToggle with next state; supports collapsible=false and testNoAnimation. • Handles null refs safely and animates body height when applicable.	
Dashboard Grid Component	• Adds the necessary styling so items are arranged in a flexible grid layout. • Displays all content you put inside it in the correct order. • Makes wider items span the full width of the grid when needed. • Keeps semantic elements like <section> and <article> unchanged.	Medium - affects how the dashboard looks and feels.
Mobile Menu Drawer Component	• Opens a slide-out menu with title and navigation items. • Highlights current selection and handles item clicks. • Includes a logout button. • Can be closed via button or programmatically.	Medium - Affects navigation and logout UX.
Toast Component	• Shows a brief message or custom content to the user. • Automatically disappears after a set time, unless disabled. • Can appear at the center or bottom of the screen. • Supports custom styles, classes, and accessibility attributes. • Handles cleanup properly so timers don't cause unexpected behavior	Medium - Improves UX, not critical if it fails.
Upload Button Component	• Renders a clickable button with customizable label and style. • Opens a hidden file picker when clicked. • Accepts only allowed file types (default image/* or custom). • Calls onPick callback with the selected file. • Clears the input after selection to allow re-uploading the same file.	Medium - Core for file upload UX, but failures don't break critical flows.
Reports Page	• Displays the Reports page inside the AppLayout. • Shows the page title "Reports". • Renders without errors or crashes.	Low - basic page rendering, important for navigation but no complex logic.
Settings Page	• Displays the Settings page inside the AppLayout. • Shows the page title "Settings". • Renders without errors or crashes.	Low - basic page rendering, essential for navigation but low complexity.

Other Utilities (Receipt Utils / Handler)	• Includes helper functions like sanitiseAmount, sanitiseDate, and other internal processing logic. • Handles data cleaning and normalisation. • Not directly exposed as APIs.	Low - Supports main functionality, failure is isolated.
---	--	--

Table 13 Tested Features

6.4. Features not to be Tested

Features that are provided by AWS services or libraries and do not require modification will not be tested.

- AWS SDK internals (e.g., S3Client, DynamoDBClient, SNSClient, TextractClient)
- AWS Lambda invocation and runtime mechanisms
- Amazon Cognito authentication and session management (login, logout, token handling)
- HTTP request/response processing provided by API Gateway (outside of custom validation logic)
- Jest or testing framework internals

6.5. Approach

The overall testing approach combined peer code reviews, automated unit testing, and manual functional testing to ensure that both backend and frontend components performed reliably and met their expected functionality. Testing was conducted iteratively throughout development to identify and address issues early, reducing integration risks and improving overall software quality.

6.5.1. Code Inspection

All code changes were subject to peer review prior to being merged into the main branch. This process ensured compliance with coding standards, logical consistency, and maintainability across the system. Automated linting and static code analysis tools were also used to detect syntax errors, unused variables, and other potential defects before runtime. These inspections acted as a preventive measure to uphold code quality and maintain consistency throughout the project.

6.5.2. Unit Testing

Unit testing formed the core of the verification process for both backend Lambda functions and frontend React components. Each Lambda function was tested under various scenarios, including valid, invalid, and edge cases, to confirm correct logic and robust error handling. Mocked AWS SDK clients were used to simulate services such as DynamoDB, Textract, and SNS, allowing accurate validation of data persistence, message publishing, and external service interaction without relying on live cloud resources.

API tests were also included as part of unit testing, using mocked HTTP events to verify endpoint behavior for GET, PUT, and OPTIONS requests. These tests ensured that requests and responses followed expected formats, that query and path parameters were correctly processed, and that invalid input was handled gracefully.

On the frontend, Jest tests confirmed that React components rendered correctly, displayed accurate data, and responded properly to user interactions.

Utility functions like `sanitiseDate`, `sanitiseAmount`, and `buildCategoryGroups` were validated against a wide range of input cases to ensure consistent and predictable behavior.

Overall, unit testing provided confidence that individual components and APIs performed reliably before integration.

6.5.3. Functional Testing

Functional testing was carried out manually to verify full end-to-end workflows across the system. Testers manually triggered S3 upload events in a test environment to confirm that the complete data flow from Textract extraction to DynamoDB storage and SNS notifications operated correctly. The resulting data was checked in DynamoDB to ensure receipt information, including vendor names, totals, and line items, was extracted and normalised as expected.

Error handling was evaluated by submitting invalid or corrupted receipts and verifying that the system logged appropriate error messages and maintained stability. On the frontend, manual testing confirmed that key features such as receipt upload, editing, and viewing performed as intended within the user interface.

In addition to manual tests, Cypress end-to-end tests were developed to automatically simulate user workflows such as login, uploading receipts, and navigating between pages, providing further assurance that core features functioned correctly in a browser environment.

6.5.4. Test Data

The test data used during development and validation consisted of real-world receipts collected by the development team. These receipts represented a variety of formats, vendors, and layouts to ensure realistic coverage of typical use cases. They were used to test data extraction accuracy and categorisation performance across different input types. Sensitive information was redacted where necessary, and the data was stored securely in accordance with project confidentiality standards.

More detailed information about the receipts used can be found in the accompanying P000340SE-EvaluationReport-Textract document.

6.5.5. Required Software

The following software is required to complete the testing:

- Node.js and Jest for unit testing
- Cypress (E2E tests)
- AWS CLI for manual testing of S3 events and DynamoDB

6.5.6. Constraints

Testing is conducted during all five sprints (2 weeks each) in parallel with development. Mocked services will be used extensively to avoid AWS service charges and minimise dependency on live infrastructure. Limited time requires selective manual testing and exploratory checks.

6.5.7. Logistics

Team of five members, each responsible for unit testing assigned modules. Integration and E2E tests conducted collaboratively. Team members will run tests on their local machines and report results through Trello and Slack.

6.5.8. Versioning

- All code in AWS CodeCommit (Frontend, Backend, Infra).
- CodeBuild/CodePipeline automate tests and deploy verified builds.
- Version tags mark stable sprint releases.

6.6. Manual Functional Test Cases

This section summarises the functional tests performed manually for key features of the system. Each test case describes the feature under test, the expected functionality, the setup and preconditions, the test data used, the expected result, and the test's priority. Manual functional testing was performed to verify end-to-end behaviour and integration with external services, as unit tests only cover isolated logic.

Test Case ID: MFTC001	
Component:	Sign-Up Page - Invalid Email
Purpose:	Validate inline email format validation
Preconditions:	User is on the sign-up page
Steps	1. Enter an invalid email (e.g., "abc"). 2. Enter any valid password and confirm password. 3. Click "Sign up"
Expected Result	Inline error message "Invalid email address" is displayed. User cannot proceed.
Priority	Pass
Priority	High
Estimated Completion Time	< 2 mins

Test Case ID: MFTC002	
Component:	Sign-Up Page - Weak Password
Purpose:	Ensure password meets complexity requirements
Preconditions:	User on sign-up page
Steps	1. Enter a valid email (e.g., "user@example.com"). 2. Enter a password that does not meet requirements (e.g., "abc123"). 3. Confirm password with the same invalid password. 4. Click "Sign up".
Expected Result	System shows validation feedback indicating missing requirements (length, number, uppercase, symbol). User cannot proceed.
Result	Pass
Priority	High
Estimated Completion Time	< 2 mins

Test Case ID: MFTC003	
Component:	Sign-Up Page - Password Mismatch
Purpose:	Ensure confirm password matches password
Preconditions:	Ensure confirm password matches password
Steps	7. Enter a valid email (e.g., "user@example.com"). 8. Enter a valid password (e.g., "Abc123!@"). 9. Enter a different confirm password (e.g., "Abc123!"). 10. Click "Sign up".
Expected Result	Inline error message "Password does not match" is displayed. User cannot proceed.
Result	Pass
Priority	High
Estimated Completion Time	< 2 mins

Test Case ID: MFTC004	
Component:	Sign-Up Page - Successful Sign-Up
Purpose:	Verify account creation and redirection
Preconditions:	User is on the sign-up page
Steps	1. Enter a valid, unused email (e.g., "newuser@example.com"). 2. Enter a valid password (e.g., "Abc123!@"). 3. Enter the same password in the confirm password field. 4. Click "Sign up".
Expected Result	New account is created. User is redirected to ReceiptCAT Dashboard page.
Result	Pass
Priority	High
Estimated Completion Time	< 3 mins

Test Case ID: MFTC005	
Component:	Sign-Up Page - Existing Email
Purpose:	Handle duplicate account attempts
Preconditions:	User is on the sign-up page; email already exists in system
Steps	1. Enter an existing email (e.g., "existing@example.com"). 2. Enter a valid password and confirm it. 3. Click "Sign up".
Expected Result	Error message "Invalid input: User already exists." is displayed. User remains on sign-up page.
Result	Pass
Priority	High
Estimated Completion Time	< 2 mins

Test Case ID: MFTC006	
Component:	Sign-Up Page - Network/Server Failure
Purpose:	Validate error handling when requests fail
Preconditions:	User is on the sign-up page
Steps	1. Enter a valid email and password. 2. Confirm the password. 3. Simulate network or server failure. 4. Click "Sign up".
Expected Result	Error message "Unable to complete sign-up. Please try again later." is displayed. User remains on current page, entered details intact.
Result	Pass
Priority	High
Estimated Completion Time	< 3 mins

Test Case ID: MFTC007	
Component:	Landing Page - View Options
Purpose:	Verify landing page elements and navigation
Preconditions:	User has navigated to the ReceiptCAT website
Steps	1. Load the landing page. 2. Observe displayed logo and tagline. 3. Check for Google login button, Email field, and "Create an Account" link. 4. Click "Create an Account". 5. Click "Sign in" to return.
Expected Result	Logo and tagline are displayed. Login options (Google button, Email field) and "Create an Account" link are visible. Clicking "Create an Account" shows sign-up form. Clicking "Sign in" returns to login view.
Result	Pass
Priority	High
Estimated Completion Time	< 3 mins

Test Case ID: MFTC008	
Component:	Upload Receipt - File Selection
Purpose:	Verify file selection dialog and supported formats
Preconditions:	User is logged in and on the Dashboard
Steps	1. Click "Upload" button. 2. Select a file that is not JPG/PNG. 3. Select a file larger than 5 MB. 4. Select a valid JPG or PNG $\leq$ 5 MB. 5. Confirm selection.
Expected Result	Files not JPG/PNG show error "Only JPG and PNG files are supported". Files > 5 MB show error "File size exceeds the 5 MB limit". Valid file loads Preview Receipt page with file displayed.
Result	Pass
Priority	High
Estimated Completion Time	< 3 mins

Test Case ID: MFTC009	
Component:	Preview Receipt - Display Details
Purpose:	Verify preview page shows correct file details
Preconditions:	User has selected a valid receipt image
Steps	1. Observe main preview area. 2. Check displayed filename and file size. 3. Verify navigation bar and global upload button.
Expected Result	Selected image is displayed. Filename and file size are correct. Global upload button is hidden. Only "Confirm Upload", "Reselect Photo", and "Back to Dashboard" buttons are visible.
Result	Pass
Priority	High
Estimated Completion Time	< 3 mins

Test Case ID: MFTC010	
Component:	Cancel Upload
Purpose:	Verify canceling the upload returns to Dashboard
Preconditions:	User has selected an image on Preview page
Steps	1. Click “Back to Dashboard” button.
Expected Result	Selected image is removed, user is returned to Dashboard. No upload occurs, no changes saved, temporary data cleared.
Result	Pass
Priority	High
Estimated Completion Time	< 2 mins

Test Case ID: MFTC011	
Component:	Reselect Image - Change or Cancel
Purpose:	Verify reselect functionality
Preconditions:	Image is displayed on Preview Receipt page
Steps	1. Click “Reselect” button. 2. Choose a different valid file. 3. Cancel file picker without selecting. 4. Choose the same file again. 5. Verify confirm upload button state.
Expected Result	Clicking reselect shows file picker. Selecting a new file updates preview. Canceling leaves original preview unchanged. Selecting same file refreshes preview without error. Confirm upload button is disabled while file picker is open. Global upload button remains hidden.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC012	
Component:	Mobile Receipt Capture
Purpose:	Verify user can capture a receipt photo directly from mobile
Preconditions:	User is logged in on a mobile device
Steps	1. On dashboard, click "Upload". 2. Allow camera permissions if prompted. 3. Deny camera permission to test fallback. 4. Take a photo. 5. Observe preview screen.
Expected Result	Camera opens (rear by default). Denying permission shows instructions and fallback to choose existing photo. Photo taken is displayed in preview screen.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC013	
Component:	Dashboard Category Totals
Purpose:	Verify totals displayed per category
Preconditions:	User has expense items assigned to categories
Steps	1. Load dashboard. 2. Observe each category card total. 3. Verify totals are sum of all item prices in that category. 4. Verify totals show exactly two decimal places.
Expected Result	Category cards display accurate totals, correctly summed, formatted to two decimal places.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC014	
Component:	Password Reset
Purpose:	Verify secure password reset via email
Preconditions:	User is registered and forgot password
Steps	1. Click "Forgot Password". 2. Receive verification email. 3. Go to reset password page. 4. Enter correct code and new password. 5. Enter invalid/expired code.
Expected Result	Correct code updates password and allows login. Invalid/expired code shows error message.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC015	
Component:	Email Verification
Purpose:	Verify email verification flow
Preconditions:	User has submitted a valid sign-up form
Steps	1. Receive verification code via email. 2. Enter incorrect/expired code. 3. Click "Resend code". 4. Enter correct verification code. 5. Simulate network/server error.
Expected Result	Incorrect code shows inline error. "Resend code" sends new code with confirmation. Correct code confirms account and redirects to dashboard. Network/server error shows plain language error, code entry preserved.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC016	
Component:	Auto-Categorisation
Purpose:	Verify automatic assignment of receipt items to predefined categories
Preconditions:	User is logged in and uploads a receipt
Steps	1. Upload a receipt with one or multiple items. 2. Observe dashboard for assigned categories and subtotals. 3. Upload item not in predefined categories. 4. Observe “Other” category. 5. Observe item count at top of dashboard.
Expected Result	Items assigned correctly to one of 25 predefined categories. Subtotals updated correctly. Items not matching predefined categories go to “Other”. Dashboard item count increases correctly.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC017	
Component:	Dashboard Category Display
Purpose:	Verify expense items are grouped and displayed correctly
Preconditions:	User has categorized expense items
Steps	1. Load dashboard. 2. Observe items under each category. 3. Expand/collapse category headers. 4. Observe items with long names. 5. Check categories with no items.
Expected Result	Items grouped under single category, not shown elsewhere. Categories display consistent label and icon. Categories with no items are hidden. Long names wrap gracefully, prices aligned right and formatted to two decimals. Category expand/collapse works correctly.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC018	
Component:	Receipt Details Page
Purpose:	Verify detailed view of a selected receipt
Preconditions:	User is logged in and has uploaded receipts
Steps	1. Navigate to Receipt History page. 2. Select a receipt from the list. 3. Scroll through the page if many items exist.
Expected Result	Receipt details page displays date, vendor, total amount, items bought, category, and item prices. Full list of items scrolls correctly without losing receipt summary at the top.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC019	
Component:	Receipt History Page
Purpose:	Verify list of receipt summaries
Preconditions:	User is logged in
Steps	1. Navigate to Receipt History page. 2. Observe displayed receipts.
Expected Result	Receipt summaries are displayed in order of most recent first. If no receipts exist, a message "No receipts available" is shown.
Result	Pass
Priority	High
Estimated Completion Time	< 4 mins

Test Case ID: MFTC020	
Component:	Receipt Image Editing (Crop & Blackout)
Purpose:	Verify user can crop or blackout receipt images before upload
Preconditions:	User has uploaded a receipt and is on the Preview page
Steps	1. Click “Crop” button. 2. Select crop area and confirm. 3. Click “Blackout” button. 4. Draw blackout areas and confirm. 5. Click “Reset” to revert edits. 6. Resize window or confirm edits. 7. Simulate cropper initialisation failure. 8. Simulate crop export failure.
Expected Result	Crop mode allows selecting area; blackout mode allows masking sensitive info. Crop/blackout buttons disabled during action. Reset restores original image. Toolbar remains responsive. Initialisation failure shows descriptive message with Reselect enabled. Crop failure shows error toast and retains current image/edit state.
Result	Pass
Priority	High
Estimated Completion Time	< 7 mins

Test Case ID: MFTC021	
Component:	Receipt Upload Progress
Purpose:	Verify upload progress, success, and failure handling
Preconditions:	User has selected a receipt image in Preview page
Steps	1. Click “Confirm Upload”. 2. Observe loading state. 3. Simulate successful upload. 4. Simulate upload failure. 5. Attempt navigation during upload.
Expected Result	“Uploading” message or spinner displayed. Successful upload redirects to Receipt History. Failed upload shows clear error with retry option. Navigation prevented during upload, with appropriate warning message.
Result	Pass
Priority	High
Estimated Completion Time	< 5 mins

Test Case ID: MFTC022	
Component:	Receipt Item Editing & Category Assignment
Purpose:	Verify user can edit item details and assign categories
Preconditions:	User is logged in and viewing a receipt with one or more items
Steps	1. Click "Edit Items" on the receipt details page. 2. Modify item fields (name, quantity, price). 3. Add a new item using "+" button. 4. Delete an item using the trash icon and confirm/cancel. 5. Open an item's category selector and choose a new category. 6. Attempt invalid category selection (should be restricted to predefined list). 7. Click "Save" to apply changes. 8. Click "Cancel" or attempt to navigate away with/without unsaved changes. 9. Simulate save failure (network/backend error).
Expected Result	Edited fields reflect changes. New item added, deleted item removed only if confirmed. Category selection restricted to predefined list, updated correctly. Save applies changes, returning view to non-editable mode. Cancel discards changes if confirmed. Errors displayed on save failure; unsaved changes remain editable.
Result	Pass
Priority	High
Estimated Completion Time	< 10 mins

Test Case ID: MFTC023	
Component:	Receipt Image Display
Purpose:	Verify uploaded receipt images are displayed correctly
Preconditions:	User is logged in and navigating to receipt details page
Steps	1. Open receipt details page with uploaded receipt image. 2. Observe image display. 3. Simulate missing or corrupted image.
Expected Result	Image is displayed clearly with no distortion. If image is missing, corrupted, or access denied, fallback message "Image not available" appears within 2 seconds.
Result	Pass
Priority	High
Estimated Completion Time	< 3 mins

Test Case ID: MFTC024	
Component:	Dashboard Visualization, Monthly & Time Range Analysis
Purpose:	Verify dashboard charts, totals, and expense items reflect correct data by category and time frame
Preconditions:	User is logged in and has uploaded receipts/expense items
Steps	1. Load dashboard page. 2. Observe expense chart for current month. 3. Hover over chart categories to verify totals and percentages. 4. Select previous or next month via chevrons or month selector. 5. Observe total expenses, category breakdown, and item listings. 6. Click a category to expand/collapse items. 7. Select custom time range via date picker. 8. Simulate backend/network errors while fetching chart data
Expected Result	Dashboard displays total expenses and charts per category accurately. Hover shows totals/percentages. Previous/next month selection updates totals, charts, and items. Categories toggle expanded/collapsed correctly. Custom time range updates charts correctly. If no data, appropriate "No data available" message shown. Errors display clear messages with fallback state.
Result	Pass
Priority	High
Estimated Completion Time	< 12 mins

Development Guide

7. DEVELOPMENT GUIDE

7.1. Project summary

ReceiptCAT is a web-based expense management prototype developed for Shine Solutions. It enables users to upload receipt images, automatically extract text using AWS Textract, classify items with AWS Bedrock, and visualise their spending distribution through an interactive dashboard.

The project demonstrates a serverless architecture using React (TypeScript) for the frontend, AWS Lambda for backend logic, and Amazon DynamoDB + S3 for data storage. Infrastructure and CI/CD pipelines are defined through AWS CloudFormation, CodeBuild, and CodePipeline, ensuring automated, reproducible deployments. Testing is conducted using Jest and Cypress, and all code is maintained in AWS CodeCommit repositories.

7.2. Setup client/ developer machine

All developers work on macOS or Windows 10+ workstations with a consistent setup based on Node.js and AWS CLI to ensure compatibility across frontend and backend environments.

Tool	Version	Purpose
Node.js	20+	Run React app and build frontend
npm	10+	Package management
AWS CLI	2.0+	Manage AWS deployment
TypeScript	5.0+	Type-safe codebase
Visual Studio Code	-	IDE for both frontend and backend
Git	-	Version control
Jest / Cypress	Latest	Unit and integration testing

Table 14 Machine Setup Version

1. Clone the repositories from AWS CodeCommit
2. Install dependencies (or run the `npm ci` command to install all the dependencies in package.json)
3. Configure environment file `.env`
4. Test the setup. Run `npm run dev` and visit <http://localhost:3000> to confirm successful setup

7.3. Code/ comment standard

- **Naming:** camelCase for variables/functions; PascalCase for React components.
- **Formatting:** Code must pass ESLint and Prettier checks before commit.
- **Function Length:** 90% functions/ components $\leq$ 250 lines.
- **Comments:** Explanatory comments were added throughout the codebase to codebase to clarify the logic, data flow and purpose of important functions and components.
- **Testing:** Each function/component must have unit tests to maintain $\geq$ 85% coverage.
- **Folder Structure:** Follow MVC pattern with src/pages, src/components, src/features and src/lib.

7.4. Version control policy

- **Repository:** Three AWS CodeCommit repos(frontend, backend, infra)
- **Branching Model:**
 1. Main $\rightarrow$ production-ready
 2. develop $\rightarrow$ integration branch
 3. feature/* $\rightarrow$ each feature or bug fix
- **Pull Request Rules:** Require at least 1 review + successful CI build
- **CI/CD:** CodePipeline auto-builds and deploys on develop/main

7.5. Deployment

The project uses AWS CodePipeline for automated deployment. When a commit is pushed to develop/main branch, CodeBuild run unit tests, Cypress and builds artifacts, then deploys frontend to S3 + CloudFront and backend Lambdas via CloudFormation.

The following sections describe the frontend, backend, and infrastructure deployments in a logical order for clarity. In practice, infrastructure must be deployed first using AWS CDK to provision resources, followed by backend and then frontend deployments.

- **Prerequisites**

1. AWS account access under Shine Solutions Group
2. IAM credentials with permission for S3, CloudFront, Lambda, API Gateway and CloudFormation
3. AWS CLI, Node.js and npm installed, Node.js $\geq$ 20.x
4. AWS CDK v2 installed globally: `npm install -g aws-cdk`
5. One-time CDK bootstrap per AWS account/region: `cdk bootstrap aws://<ACCOUNT_ID>/ap-southeast-2`
6. Ensure both frontend and backend .env files are properly configured with valid AWS environment variables.

- **Frontend Deployment**

1. Build the project: `npm run build`
2. Hosted URL: <https://dewsori2weo7.cloudfront.net>

- **Backend Deployment**

1. Build backend functions: `npm run build`
2. Deploy using CloudFormation or the existing AWS CodePipeline: `cdk deploy`
3. API Endpoint: <https://gnlh56t324.execute-api.ap-southeast-2.amazonaws.com/dev/>

- **Infrastructure Deployment**

1. Set environment variables:
 - (1) `export CDK_DEFAULT_ACCOUNT=<ACCOUNT_ID>`
 - (2) `export CDK_DEFAULT_REGION=ap-southeast-2`
 - (3) `export FRONTEND_URL_DEV="http://<prod-cloudfront-domain>"`
 - (4) The value of `FRONTEND_URL_DEV` should be replaced with the actual CloudFront domain generated after the first frontend deployment (for example, <http://dewsori2weo7.cloudfront.net>).

2. Deploy core Infra:

- (1) `cdk deploy \`
- (2) `receiptcat-common \`
- (3) `receiptcat-backend-dev/infra \`
- (4) `receiptcat-backend-dev/pipeline \`
- (5) `receiptcat-backend-dev \`
- (6) `receiptcat-frontend-dev/infra`

3. Update the `FRONTEND_URL_DEV` variable with the actual CloudFront domain

4. Configure Cognito Hosted UI callback URLs and re-deploy backend/frontend infra for CORS and SSM updates.

- (1) `cdk deploy receiptcat-backend-dev/infra`
- (2) `cdk deploy receiptcat-frontend-dev/infra`

5. Deploy frontend pipeline and wrapper stacks: `cdk deploy receiptcat-frontend-dev/pipeline receiptcat-frontend-dev`

- (1) Production Deployment: For demonstration, only the development environment (develop branch) has been deployed. To deploy the production environment, repeat the same steps using production stacks: `export FRONTEND_URL_PROD="https://<prod-cloudfront-domain>"`
- (2) `cdk deploy receiptcat-backend-prod/* receiptcat-frontend-prod/*`
- (3) Once deployed, a new CloudFront and API Gateway endpoint will be generated. Ensure Cognito Hosted UI URLs are updated for production domains

6. Testing the Deployed App

- (1) Open the CloudFront URL (for example: <https://dewsori2weo7.cloudfront.net>)
- (2) Sign in/Sign up using a Cognito user account
- (3) Upload a test receipt and verify the following:
 - The image appears in the S3 bucket
 - Extracted and categorised data is stored in DynamoDB
 - The uploaded receipt and its items are visible in the web dashboard/receipt history page
- (4) Check AWS CloudWatch logs for Lambda execution success and API Gateway response(HTTP 200).

7. Successful deployment can be verified when:

- (1) The application loads correctly in CloudFront.
- (2) User authentication works through Cognito
- (3) Uploaded receipts are processed, and items are displayed in the dashboard/history page
- (4) DynamoDB and S3 data are updated without error.

7.6. Getting started

This section demonstrates how one of the key features, Receipt Upload, operates across the frontend and backend and underlying infrastructure, including setup, implementation, and deployment.

The feature runs on AWS serverless architecture provisioned by AWS CDK, which includes S3 for image storage, API Gateway and Lambda for processing, DynamoDB for data persistence, and AWS Textract and Bedrock for AI-powered extraction and categorisation.

7.6.1. How to start

After logging in via AWS Cognito, users navigate to the Upload Receipt page. They can select a receipt image (JPG/PNG) and click Confirm Upload to begin the process. The system then requests a presigned S3 URL from the backend and uploads the image directly to Amazon S3.

7.6.2. Frontend Implementation

The frontend is built in React (TypeScript) and handles image preview, validation, and upload logic.

1. The component calls a backend API endpoint *POST/upload/presign* to request a presigned S3 URL.
2. Once received, the image is uploaded directly to Amazon S3 via HTTPS, bypassing the backend to improve speed.
3. Upload progress and success/error notifications are displayed to the user.

7.6.3. Backend Implementation

The backend runs on AWS Lambda (TypeScript), exposed through API Gateway.

1. The Lambda function validates the user's Cognito token via the API Gateway Authorizer.
2. It generates a presigned S3 URL and object key under the user's namespace, valid for 30 seconds
3. The response returns a JSON object containing the presigned upload URL and the S3 key:
4. The frontend then uses this `uploadUrl` to perform a direct file upload to S3.

REFERENCES

8. REFERENCES

1. Amazon Web Services (2021) *Uploading to Amazon S3 directly from a web or mobile application*, AWS Compute Blog, accessed 25 August 2025.
<https://aws.amazon.com/blogs/compute/uploading-to-amazon-s3-directly-from-a-web-or-mobile-application/>
2. Amazon Web Services (2024) *Amazon Bedrock – General Reference*, AWS Documentation, accessed 31 August October 2025.
<https://docs.aws.amazon.com/general/latest/gr/bedrock.html>
3. Amazon Web Services (2024) *Amazon Bedrock Pricing*, AWS website, accessed 31 August 2025. <https://aws.amazon.com/bedrock/pricing/>
4. Amazon Web Services (2024) *Amazon Cognito user pools identity federation*, AWS Documentation, accessed 27 August 2025.
<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-identity-federation.html>
5. Amazon Web Services (2024) *Amazon Cognito user pools managed login*, AWS Documentation, accessed 5 September 2025.
<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-managed-login.html>
6. Amazon Web Services (2022) *Unit testing AWS Lambda with Python and mock AWS services*, AWS DevOps Blog, accessed 16 September 2025.
<https://aws.amazon.com/blogs/devops/unit-testing-aws-lambda-with-python-and-mock-aws-services/>
7. Amazon Web Services (2024) *AnalyzeExpense – Amazon Textract API Reference*, AWS Documentation, accessed 22 September 2025.
https://docs.aws.amazon.com/textract/latest/dg/API_AnalyzeExpense.html
8. Amazon Web Services (2024) *AnalyzeDocument – Amazon Textract API Reference*, AWS Documentation, accessed 22 September 2025.
https://docs.aws.amazon.com/textract/latest/dg/API_AnalyzeDocument.html
9. Paulo G (2017) *Unit Testing and the Arrange, Act and Assert (AAA) Pattern*, Medium website, accessed 10 September 2025. <https://medium.com/@pjbfgf/title-testing-code-ocd-and-the-aaa-pattern-df453975ab80>